

1.python 基础

*python 熟悉吗？

1.还行, 很简单, 用于做自动化测试是没有问题的, python 是面向对象的弱类型语言(封装、继承、多态), 支持面向对象编程, 也支持传统的函数式编程。

2.我们之前的 python+requests 接口自动化测试用例的分层就要使用 class 类封装方法, 根据类创建对象, 调用对象中的方法。

3. (如果面试官没有打断, 接下来就跟面试官说一下 python+requests 做接口测试的流程)

PS : 1.python3.6、python3.7、python3.8 目前是常用的版本。

2.模块、库、组件、框架是近义词。最简单的模块就是一个.py 文件。

3.python 是解释性语言, 不需要编译直接运行。java 和 c/c++ 都是需要编译后, 才能执行的。

*会写脚本吗？

会, 这个简单, 之前写过接口自动化测试框架, 忘记了还能套用之前的模板或者百度。(代码和.jmx 文件可以称为脚本)

*unittest 框架了解吗？

unittest 框架是 python 自带的模块, 不需要通过 pip install 下载安装。

使用步骤：

- 1.自定义类来继承 unittest.TestCase 测试用例类；
- 2.复写 setUp()/tearDown()方法, 并且定义 test 方法；
(执行顺序是 setUp() -> testxx -> tearDown())
- 3.写主程序, 执行 unittest.main()方法, 则会进行测试。

特点：

- 1.setUp() / setUpClass() --环境预设, test 方法执行之前运行；
- 2.testCase() --测试用例；
- 3.tearDown() / tearDownClass() --环境恢复, test 方法执行之后运行。

unittest 还可结合 ddt 进行数据驱动测试, 使用断言来判断用例是否通过, 生成测试报告。

单元测试框架介绍：

- 1.unittest : python 自带的单元测试框架。测试用例类必须继承父类 unittest.TestCase, 且测试方法名必须以 test 开头, 不需要执行的测试方法(测试用例)可以使用装饰器@unittest.skip。用例执行顺序是方法名的字母顺序。
- 2.pytest : 需要手动安装的第三方单元测试框架, 属于非侵入式设计。测试类所在的 py 文件必须 test_ 开头, 测试用例类必须 Test 开头, 测试用例方法必须 test_ 开头。pytest 还可以执行 unittest。

*setUp(), tearDown()和 setUpClass(), tearDownClass()的区别：

参考答案：

- 1.当 test 测试用例方法有 n 个, setUp()和 tearDown()就会执行 n 次。
- 2.不管测试用例方法有多少个, setUpClass()和 tearDownClass()只会执行一次且需要添加装饰器@classmethod。
- 3.setUpClass()在 setUp()之前先执行, tearDownClass()在 tearDown()之后执行。

PS : 1.在 web 自动化：环境预设主要指打开浏览器, 访问目标页面, 设置一些公共信息。环境恢复主要指关闭浏览器。

2.在接口自动化：环境预设主要指创建本地会话对象，设置一些公共信息。环境恢复主要指关闭本地会话对象。

range()对象的作用

生成一个范围整数，

比如 range (1,101) 表达 1 到 100 的值（算头不算尾），

range (101) 表达 0 到 100 的值（算头不算尾）。

PS：还可以设置 step 步长。

pass 语句的作用

凑格式、占位符，当类体或方法体暂时没有代码时，防止出现语法错误。等价于 ...

return 返回和 print()打印的区别

函数(方法) 或 变量名返回的值是躲在内存里面的。

如果想用眼睛看到，则需要通过 print()方法输出打印到显示器。(返回和打印是两个不同的概念)

*python3 的数据类型，哪些是可变的，哪些是不可变的？

数据类型：

1.基本数据类型：int：整型，float：浮点型，str：字符串，bool：布尔型

2.数据结构(容器类型)：list：列表，tuple：元组，dictionary：字典，set：集合

可变：list，dictionary，set（记住可变的即可，其他的就是不可变）

不可变：int，float，str，bool，tuple

*列表和元组的区别？

列表：元素是可以变化的。

元组：元素是不可变的(不可以修改)。

总结：

1) 列表可变，元组不可变

2) 列表可增删改查，元组只能查询

*列表如何添加元素？

list.append()方法；移除元素 list.remove()；修改元素：list[1] = 'jackie'

*列表如何去重

```
my_list = [1,3,4,4,6,6]
```

1. 把列表转为 set 集合，就可以去除列表的重复元素。

```
myset = set(my_list)
```

2. 或者 使用 for 循环遍历每个元素，判断当前元素的数量是否大于 1，如果是则调用列表的 remove 方法就行。

```
my_list = [1,3,4,4,6,6]
for i in my_list:
    if my_list.count(i)>1:
```

```
my_list.remove(i) # 只移除第一次出现的i 值
print(my_list)
```

PS : 类型转换 : `str(值)` : 转为字符串 ; `list(set)` : 集合转为列表 ; `set` 集合用大括号 `{}` 表示 , 空集合是 `set()`。
`set` 集合的特点的无序无重复性 (两无) 。

*列表常见的方法(增删改查)

`list.append(obj)` # 添加列表元素 (在末尾追加)

`list.insert(2,77)` # 在指定索引处插入值

`list.remove(obj)` # 移除列表元素

`list.pop(-1)` # 指定索引删除

`list[2]=88` # 在指定索引修改值

`a = list[2]` # 获取指定位置的元素 , 从 0 开始编号 , -1 代表倒数第一个

`list.__len__()` # 获取列表长度

`list.reverse()` # 反转列表

`list.sort(key=None, reverse=False)` # 列表排序(默认正序)

*列表如何排序

调用列表的排序方法 `list.sort()` 或 内置的排序函数 `sorted(list)` , 还可手写 “选择排序” 或 “冒泡排序” 的算法。

PS : `list.sort(key=lambda x:len(x), reverse=False)` # 返回 `None` 值。 (或者 `key=len`)

`sorted(list, key=lambda x:len(x), reverse=True)` # 返回排序后的新列表。 (或者 `key=len`)

`reverse` 默认为 `False` 升序, `True` 为降序

`key` 是排序的关键字, 默认根据元素的大小排序 (`key=lambda x:x`)

`key` 记录一个匿名函数, 遍历到每个元素都会执行一下该匿名函数。 `lambda x:len(x)` 是根据元素的长度排

*python 中如何切片(如何获取字符串或列表的部分元素)

使用中括号和冒号获取一部分元素 (算头不算尾) , 比如 `str[a:b]`、`list[a:b]`。 (`str.split()` 切割得到的是列表)

*字典如何添加元素 (增删改查) ?

根据 “中括号键” 等于值来添加 : `info_dict['age'] = 18` (或方法 `info_dict.__setitem__('age', 18)`)

根据键删除元素 : `info_dict.pop('age')`

根据键值对修改 : `info_dict['age'] = 100`

获取元素 : “中括号键” 获取值 : `name = dict['key']` 或 调用字典的 `get` 方法获取值 : `dict.get('key')`

`dict.keys()` # 获取所有键 (列表)

`dict.values()` # 获取所有值 (列表)

`dict.items()` # 获取键值元组的列表

*字典和 json 的区别

- 1.字典是 python 中的一种数据类型，而 json 是字符串（网络传输的数据都是字符串形式）；
- 2.字典的键值对，可以使用单或双引号。而 json 中的键值对只能使用双引号。

PS : `json.dumps(dict)` : 将字典转换为 JSON 格式的字符串。

`json.loads(json_str)` : 将 JSON 格式的字符串转化为字典。

*字典和列表(元组)的区别

- 1.字典无序且键不能重复（两无，无序无重复），列表(元组)有序且可以重复（两有，有序有重复）。
- 2.字典和列表都可变。（元组不可变，set 集合可变）

PS : 列表(元组)的区别：set 集合无序无重复（两无），列表(元组)有序有重复（两有）。

*unittest 单元测试框架，由哪三部分组成？

unittest 框架，由 `setUp()/setUpClass()`--环境预设，`testCase()`--测试用例
`tearDown()/tearDownClass()`--环境恢复，三大部分组成。

unittest 在接口自动化应用（基于单接口测试的举例）：

- 1.在 `setUp` 中创建本地会话对象、定义请求 ip、请求头信息。在 `testCase` 中构造请求 url 和请求参数。
在 `tearDown` 中关闭本地会话对象。
- 2.unittest 框架可组织执行测试用例且生成 HTML 测试报告。
- 3.unittest 提供丰富的断言方法。

unittest 框架下的断言方式有哪些？

断言方式：我项目中常用的断言方法是：`assertEqual`（等于）和 `assertIn`（包含）；自动判断测试用例是否通过，最终生成测试结果。

*unittest 如何生成 html 测试报告中

0.先把 `HTMLTestRunner.py` 放到 python 安装根目录 Lib 下

- 1.创建测试集
- 2.定义测试报告文件的路径
- 3.创建 html 运行器（对象），且封装测试报告文件对象，最后调用运行器的 `run(test_suit)`方法即可

*unittest 如何进行参数化

- 1.先在继承了 `unittest.TestCase` 的测试类上声明装饰器 `@ddt.ddt`
- 2.然后在测试方法上声明 `@ddt.data()` 或 `@ddt.file_data()` 和 `@ddt.unpack`

PS : `ddt` : data driver test : 数据驱动测试，指参数化。刚好也是某个参数化模块的名字。

在脚本中，怎样捕捉异常信息

- 1.使用 `try except` 语句即可捕捉异常信息。
- 2.具体格式：

`try:`

可能出现异常的代码

except :

出现异常后处理的代码

PS : 1.完整的异常处理代码 : try except finally。异常处理的作用 : 提高程序的健壮性、容错性。

2.finally 是最终一定执行的 ,raise 可以手动抛出异常。抛出异常可看成是返回特殊值 ,当前函数体会中断 ,除非使用 try except 捕捉该异常。

*python 中类属性和对象属性的关系

类中属性是所有对象共享的属性 ,对象属性仅是当前对象的属性。

全局变量、局部变量、参数、属性的区别

全局变量定义在 py 文件中 ,和 class 类同层次 ,所有位置都可以调用。

局部变量在函数体或者方法体定义 ,只能在当前局部调用。局部位置可以通过 global 关键字来声明全局变量。

参数是定义在函数的参数列表中 ,只能用于当前函数体 ,属于特殊的局部变量。

属性是类的成员变量或对象的成员变量。self.属性名是当前对象的属性 ,cls.属性名是当前类的属性。

PS : 形参 : 参数名 ; 实参 : 参数值。

*装饰器原理是什么 , 说出你了解的常见装饰器

1.装饰器是使用@符号开头的特殊函数或类 ,用于装饰某方法 ,为该方法额外添加代码 ,修改该方法的功能 ,有助于让我们的代码更简洁。(高内聚低耦合)

2.常见的装饰器@classmethod 类方法、@staticmethod 静态方法 ,他们修饰的方法都可以使用类名来调用 ,也可以使用对象来调用。区别是@classmethod 装饰的方法会自动传入当前类对象 cls。

@unittest.skip()装饰器的功能是忽略跳过当前方法 ;

@ddt.data()装饰器的功能是实现参数化。

*匿名函数是什么

用 lambda 关键词创建匿名函数 ,这种函数没有函数名 ,用变量名记录。

定义求和的匿名函数

```
sum = lambda a, b: a + b
```

调用匿名函数

```
print( sum(1,2) ) # 结果 3
```

*类和对象的关系

1.类是描述 ,对象是实体 (实例) ,通过对象可以调用方法。

2.类中属性是所有对象共享的。

PS : 创建对象的细节 : 通过类名+括号创建对象 Dog() ,先调用__new__()方法创建对象 ,再调用__init__()初始化方法为对象赋属性值。

*面向对象的特点(什么是面向对象)

面向对象有封装、继承、多态。封装指类封装了属性和方法, 继承指子类继承父类, 通过子类对象可以拿到当前父对象的成员。Python 的多态比较弱, 崇尚“鸭子类型”, 因为 Python 是弱类型语言。(java 能完美体现多态的特点)

*什么是方法的重写(复写、覆盖)

如果父类的同名方法不能满足子类的需求, 可以对其进行复写。复写后, 按照就近原则调用子类对象的该同名方法。
(子类属性同理)

函数和方法的区别

1. 在 class 类中定义的函数叫方法, 类外定义的函数叫函数。

2. 方法与函数的特别区别: 方法第 1 个参数名称默认为 self (可以换名)。

PS: 1. 在 java 中函数和方法的名称不区分, 两者是 synonym。java 是完全面向对象的语言。

2. 单纯调用方法名(函数名), 没有加括号, 其实不会执行该方法, 返回的是该方法的方法体对象。

*如何利用 python 生成大量数据(比如 1w 个手机号、用户名)

使用 for 循环来拼接手机号(用户名), 循环体外先使用 open()方法并设置成 w 模式, 在循环体不断写入内容。

PS: 循环体结束后, 需要调用 file.flush()方法和 file.close()方法。

*怎样用 Python 连接 mysql 数据库?

1. 先安装 pymysql 模块。

2. 再调用 pymysql.connect()连接方法并传入数据库 ip、端口、用户名、密码、数据库名。

3. 之后创建 cursor 游标对象, 调用游标对象的 excute()方法并传入 sql 语句。数据库响应的结果集封装到游标对象中。

PS: Oracle 同理。其实工作中一般使用 sqlyog、navicat 连接 mysql 数据库。

*字符串有关方法(字符串是不可变)

```
# 1. 如何查找字符串中某元素的个数, 比如'hello'中'l'的个数
str0 = 'hello'
print(str0.count('l')) # 2
# 2. 如何查找字符串中某元素的索引, 比如'hello'中'e'的索引数
print(str0.find('e')) # 1
# 3. 如何统计字符串或列表或元组的长度(大小): 对象 obj.__len__() 方法或 s 使用 len(obj) 函数
print(str0.__len__()) # 5
print(list(str0).__len__()) # 5
print(tuple(str0).__len__()) # 5
# 4. 如何获取一个字符串中的目标值, 比如登录成功后返回了包含 token 的 json 数据, 如何获取 token 值呢?
# 可以使用 str.split() 切割方法, 针对 token 值的左右边界字符串切割 2 次即可。
# 或者把 json 字符串转为字典后, 根据字典的键 token 来获取值

# 5. 一个字符串'hello world'中的空格如何去掉: 用空字符串代替空格即可
str1 = 'hello world'
str2 = str1.replace(' ', '')
```

```

print(str2)

# 6. 一个字符串如何去掉头尾的空格: 调用字符串的 strip() 方法(或用空字符串代替空格)
str1 = ' helloworld '
str2 = str1.strip()
print(str2)

# 7. 列表如何转为字符串: 通过 for 循环遍历每个元素并转为字符串, 且和空字符串拼接即可。
li = [1,2,3,'h','ee']
st = ''
for i in li:
    st = st+str(i)
print(st)

# 8. 列表如何翻转: 调用列表的 list.reverse() 方法 或 list[::-1]
li.reverse()
print(li) # 变化原列表

# 9. 字符串如何反转, hello 变 olleh: 使用反向切片 str[::-1] 即可
# 或者先把字符串转为列表并反转, 然后调用空字符串的 ''.join(list) 方法
# 第 1 种
str1 = 'hello'
print(str1[::-1])
# 第 2 种
list2 = list(str1)
list2.reverse() # 列表反转
print(''.join(list2)) # 列表转字符串

```

*冒泡排序

1. 双层 for 循环, 内循环次数越来越少。
2. 当次内循环, 相邻元素比较, 左边的大则换到右边。(升序)

```
'''
```

冒泡排序:

原理:

1. 相邻元素比较, 谁大, 谁放右边, 这样比一圈, 最右边的值就是最大值。
2. 剩下元素里面, 继续这样比, 第二个最大值放到最右边(倒数第二位)

实现:

1. 双层 for 循环, 外循环需要循环 $n-1$ 次。
2. 第一次外循环, 内循环需要循环 $n-1$ 次, 内循环次数是逐减的。

```
'''
list = [1, 5, 3, 18, 8]
for i in range(list.__len__()-1): # 外循环4次
    for j in range(list.__len__()-1-i): # 一开始, 内循环4次
        if list[j] > list[j+1]:
            # 相邻元素比较, 如果左边的大, 则互换位置, 保证右边的值要大
            list[j], list[j+1] = list[j+1], list[j]
# 循环结束
print(list)
# 定义函数
def bubble_sort(list):
    for i in range(list.__len__()-1): # 外循环4次
        for j in range(list.__len__()-1-i): # 一开始, 内循环4次
            if list[j] > list[j+1]:
                # 相邻元素比较, 如果左边的大, 则互换位置, 保证右边的值要大
                list[j], list[j+1] = list[j+1], list[j]
    return list
# 调用函数
list = [1, 5, 3, 18, 8]
print( bubble_sort(list=list) )
# 调用龟叔写好的排序方法
list = [6,4,9,3,2]
list.sort()
print(list)
```

*九九乘法口诀表

1. 双层 for 循环, 内循环次数越来越多。
2. 当次内循环, 拼接一下 i 乘以 j 等于 i*j 即可。

```
# 打印99乘法表
for i in range(1, 10):
    for j in range(1, i + 1):
        print(f'{i}*{j}={i*j}', end=" ") # 同行打印, 不换行
    print(); # 当次换行
```


***求 $1 + 2 + 3 + \dots + 100$ 的和?**

1.如果使用 while 循环, 循环体核心表达式有两句: $sum = sum + i$; $i = i + 1$

2.通过循环语句控制循环 100 次即可: $i \leq 100$ 。(然后 i 和 sum 的初始化值设为 0)

```
# 使用while 循环, 求1 到100 的和。

sum = 0;
i = 0;
while i <=100:
    sum = sum + i; #0+0, 0+0+1, 0+0+1+2, ...
    i = i + 1; # i 的值不断变大

# 循环结束后
print(f'sum={sum}')
```

```
# 使用for 循环, 求1 到100 的和。

sum = 0;
for i in range(1,101):
    sum = sum + i; #0+1, 0+1+2, 0+1+2+3, ...

# 循环结束后
print(f'sum={sum}')
```

***判断三角形**

```
# 从键盘上输入三个整数值, 作为三个边长a,b,c,

# 判断并打印出5 种可能: 1. 边长不合法; 2. 等边三角形; 3. 等腰三角形; 4. 普通三角形;
5. 非三角形;

a = int(input('请输入边长 a:'))
b = int(input('请输入边长 b:'))
c = int(input('请输入边长 c:'))
if a<=0 or b<=0 or c<=0:
    print('边长不合法...')
else:
    if a+b>c and a+c>b and c+b>a:
        if a==b==c:
            print('等边三角形')
        elif a==b or b==c or a==c:
```

```
        print('等腰三角形')
    else:
        print('普通三角形')
    else:
        print('非三角形')
```

***代码分层的好处 (po 模式 和 api 接口分层)**

降低程序的耦合度, 提高程序的扩展性和可维护性。

***如何往文件中写入内容(如何把结果打印到文件中)**

很简单, 使用 open()方法创建文件对象, 设置模式为 w 可写, 调用对象的 write()方法写入内容。

如果是追加内容的话, 模式设为 a 追加。(默认情况下为 r 可读)

PS : file=open(file='文件路径', mode='w'); file.write(s)

2.python+selenium的web自动化(非重点、不要背)

*自动化测试怎么做的 或 自动化测试流程

就拿简历上的 xxx 项目来说：

1. 系统稳定后，我们先对系统进行评估，确认这个系统是否可以实现 UI 自动化。
挑选能实现自动化测试的重要功能用例(优先考虑主流程)，用于回归老模块的自动化测试。
(可能是每个人都负责各自模块的自动化，也可让厉害的人单独负责)
2. 我们是用 python+selenium+pytest+allure 工具来实现自动化。比如，下单这个功能的脚本，我们是这样做的：首先，我们会创建一个 python 测试项目(自动化测试框架)，利用 PO 模式+三层结构，实现 page 页面层、business 业务层、testcase 测试用例层。每个 Page 类都有对应的 Business 类和 Testcase 类。Page 类封装了页面元素的 set、click 等操作方法。Business 类封装了页面的完整业务方法。Testcase 类封装了针对页面业务的测试方法(添加参数化、断言、异常处理、日志处理)。
还分 data 数据层(存放测试数据)，log 日志层，img 截图层，report 测试报告层。tools 工具类层、common 通用方法层。
3. 脚本调试成功后，我们还可以用 jenkins 持续集成工具，设置定时跑脚本，自动生成 html 测试报告。
并自动发送邮件给相关人。

PS：1.一般金融项目变化快，ui 自动化相对来说做的少点。金融项目以功能测试和接口测试为主。

因为银监会、证监会、保监会经常发布新的国家金融政策，所以金融项目会进行相应的修改。

2.管理类软件，比如 erp、crm、oa 系统，一般在公司企业内部使用，变化比较慢，比较适合做自动化。

3.新项目不可能做自动化的，因为功能不稳定，经常变。

等版本稳定了才考虑做自动化，自动化主要用于回归测试(老模块的回归测试)。

4.一般让 python (java) 代码能力比较强的人做自动化。

*自动化熟悉吗？

还行，很简单，我会 python+selenium 的 web 自动化。吹一波“python+selenium 的 web 自动化测试框架的搭建”。

PS：1.学的好的同学，就说自己可以独立完成 web 自动化测试框架的搭建。

2.没什么信心的同学，就说自己可以套用其他项目的 web 自动化测试框架来完成。

毕竟工作是开卷的，可以翻笔记资料，还可以百度。

*python+selenium 的 web 自动化测试框架如何搭建

1. 我们是用 python+selenium+pytest+allure 工具来实现自动化。利用 PO 模式+三层结构，实现 page 页面层、business 业务层、testcase 测试用例层。每个 Page 类都有对应的 Business 类和 Testcase 类。Page 类封装了页面元素的 set、click 等操作方法。Business 类封装了页面的完整业务方法。Testcase 类封装了针对页面业务的测试方法(添加参数化、断言、异常处理、日志处理)。
2. 还有 data 数据层(存放测试数据)，log 日志层，img 截图层，report 测试报告层。tools 工具类层、common 通用方法层。
3. 脚本调试成功后，我们还可以用 jenkins 持续集成工具，定时跑测试脚本，生成 html 测试报告，并自动发送邮件给相关人。

***定位元素有哪几种方式**

8 大方式：id、name、class_name、tag_name、link_text、partial_link_text、xpath、css_selector
常用的是 id 和 xpath 定位。（selenium 4.0 以后只保留了一个方法）

xpath 和 CSS 定位方式的区别：

- 1、xpath 是根据路径定位，有相对路径和绝对路径，定位功能很强大。
- 2、CSS 语法更简洁且定位比较快，有 id 定位、属性定位、标签定位，层次定位等。

***自动化率多少？**

一般是 30% 到 50%，这个没有固定的，我们是把优先级高的功能用例转为自动化脚本用例。

比如，冒烟测试的测试用例可以转换成自动化脚本。

回归测试的老模块用例也可以转，重要的新模块用例也可以转。

写过多少条自动化测试用例？

这个没有具体算过。只要有时间，稳定的模块功能都会写。就拿上个项目来说，UI 自动化测试用例大概写了将近有 70、80 条左右这样子吧。

你们自动化脚本的通过率是多少？（挖坑题）

一般情况下自动化脚本都是 100%运行通过。

如果后端环境出现了问题，异常情况比较多，或者开发修改了代码没通知到测试人员及时修改脚本，自动化脚本通过率就可能会低一些，但是最终都是要达到 100%通过。

PS：自动化测试目的不是为了发现 bug，是为了方便回归测试，验证没有 bug。

***自动化测试的目的（意义）或为什么要做自动化测试？**

一般用于回归测试（回归老模块），释放人的双手，提高效率。（不是为了测出 bug，场景覆盖功能的 30%-40%左右）

***做过长时间的自动化，你之前写过多少自动化测试用例？**

一直有做，主要回归测试使用自动化技术。

自动化测试用例没有具体算过，稳定的模块都会写，拿之前的 xx 项目来说，自动化测试用例大概写了将近 50 条。

你的 python 水平很一般啊？（遇到这种否定你的问题，一定不能虚！）

- 1.我现在掌握的 python 知识，做自动化测试是没有问题的，并且工作可以翻笔记和资料的；
- 2.类的封装、创建对象、方法调用这些都没问题；
- 3.我一般是会做，但不是很会用文字描述出来，我以注意到这点，现在也在加强提升自己的总结能力。

***python 的第三方模块/标准库有哪些？**

第三方库（手动下载安装的）：

selenium

requests

pytest

allure-pytest

pymysql
HTMLTestRunner

标准库 (python 自带的) :

unittest
math
random
datetime
time

webdriver 的原理是什么，也就是 selenium 实现 web 自动化的原理？

浏览器驱动，接收自动化脚本发过来的指令(指令就是我们的脚本)，浏览器的驱动根据接收到的指令，驱动浏览器工作。

你们是怎么检查自动化的结果是不是正确的？

我们会用 assert 断言来检查实际结果和预期结果是否一致。常见的断言方式主要有
assert in , self.assertEqual() , self.assertIn()。

unittest 单元测试框架怎么安装的？(挖坑题)

不需要安装，是 python 自带的，直接通过 import 关键字引用就可以了。(ddt 需要手动下载)

***隐式等待和显式等待的区别**

隐式等待是针对所有元素，显示等待是针对某个特定元素，相同点是在超时时间内获取到了该元素就立即往下走。
sleep 是强制等待，必须到了设置时间才往下走。(推荐使用隐式等待)

***po (page object : 页面对象) 设计模式了解吗？**

1.po (pom) 还是了解的，PO 模式+三层结构是 web 自动化测试代码设计的最佳的方式。

2.主要实现了 page 页面层、business 业务层、testcase 测试用例层。

测试代码框架还有 data 数据层(存放测试数据)，log 日志层，img 截图层，report 测试报告层。tools 工具类层、common 通用方法层。

(把公共代码封装到 Base 基类中，使用继承获取其中的方法)

3.如果页面元素发生了变化，只需要调整对应的 page 页面类即可，提高测试用例的可维护性。

***web 自动化常识**

1.web 自动化负责人数：一般项目组 5 个测试，很可能就 1 个人负责 web 自动化测试。或者 2 个人负责，或者大家都负责自己模块的 web 自动化测试。(web 自动化的目的不是为了发现 bug，而是主要用于回归测试)

2.一般所说的自动化，如果没有特别指明，默认指 web 自动化。ui 自动化包括 web 自动化和 app 自动化。
移动端自动化指 app 自动化、H5 手机网页的自动化、小程序自动化。

3. 写自动化脚本，可能会用到 svn 或 git 管理自动化脚本(svn 会多点，因为使用简单方便)。

特别是大家协同开发自动化脚本时，一般都要使用版本管理工具来管理脚本了。(python 接口自动化同理)

3.接口自动化

*接口测试怎么做 或 接口(自动化)测试流程

0. jmeter 接口自动化和 python+request 接口自动化我都会，我先给你介绍 jmeter 接口自动化吧。

1、需求分析后，开发在设计阶段会基于需求书编写接口文档，然后开发和前端开发、接口测试人员一起参与评审接口文档。

2、评审之后，基于接口文档并结合需求的理解而编写接口案例了（开发此时正在编写接口代码）：

2.1 了解每个接口对应的功能及项目 ip、端口号；

2.2 还有接口的请求路径、请求方式、请求参数、约束条件、加密方式；

2.3 及接口响应的“业务状态码”等等。

可以利用 xmind 和 excel 编写用例，一个“业务状态码”其实就是一个测试点了（和功能用例设计没有什么区别）

3、接口测试用例编写完成后，参与用例评审(和功能用例一起评审)，然后开发搭建好测试环境，开发协助准备测试数据。我们就可以开始测试了。

4、我们是用 jmeter 执行接口测试。

先建立一个线程组，再添加 http 请求，填写 ip、端口、请求路径、请求方式及请求参数，并添加查看结果树。还可以根据需要设置参数化，断言等。

5、（非必须步骤，可选）接口测试失败，404 或 500 那种错误先自我检查请求路径及参数是否有格式问题，修改后再进行测试。

如果确实是接口本身存在 bug，就把服务器上的日志取下来，和开发沟通后，在禅道上提 bug 单给开发修改。后面一直到接口都没问题了，就可以使用 Jenkins+Ant+Jmeter 上做持续集成的接口自动化测试(回归测试)，定时跑测试脚本，生成 html 测试报告，并自动发送邮件给相关人。

补充(加分)：(面试必须吹牛自己的自动化和接口测试做的多且很厉害，否则面试官对你没有兴趣，拿不到 offer 的)。

我也用过 python+requests+pytest+allure/python+requests+unittest+ddt 进行接口自动化测试框架开发：

1.首先，我们要构建一个测试工程，进行代码分层设计(核心：api 层 + testcase 层)：

api 接口层：定义 api 接口类，类中存放访问接口的方法。

testcase 测试用例层：定义测试类，类中存放测试接口的方法。还可以引入参数化、断言、异常处理、日志处理。

common 通用层：定义公共方法，比如获取已登录的会话对象或者 token 值。

data 数据层：基于代码和数据分离的原则，参数化的数据文件存放此处。（.yaml 文件/json 文件）

log 日志层：日志文件的存放目录。

report 测试报告层：html 测试报告存放目录。

tools 层：存放一些自定义工具类，比如 DB 数据库类，类中定义访问数据库的方法，Log 日志类，类中定义日志的处理方式，还可以定义处理文件的类。

main_test.py 主程序：执行测试用例并输出 html 测试报告。

2.脚本调试好，没有问题了，可使用 jenkins 进行持续集成的接口自动化测试，定时跑测试脚本，生成 html 测试报告，并自动发送邮件给相关人。

PS : 1.编写 jmeter 测试脚本(.jmx 文件)或者编写 python+request 接口自动化测试脚本(.py 文件)属于测试执行的阶段，而不属于前期准备阶段。

2. jmeter 和 python+requests 都属于接口自动化，工作中一般是二选一。前者是可视化的工具，简单易用。后者是脚本代码，上手难，但是可扩展性强。（自动化技术主要用于回归测试(回归老模块)）

3.接口测试流程和功能测试基本一致。工作中，jmeter 接口测试比功能测试还要简单。

4.概要设计阶段开发主要编写接口文档，详细设计阶段开发主要编写类名及方法名、设计表名及字段名。

*自动化熟悉吗(接口自动化熟悉吗)?

web 自动化了解点，使用 python+selenium 来做，八大元素定位和 po 模式。

我主要擅长 python+requests 接口自动化。吹一波“python+requests 接口自动化测试框架的搭建”。

PS : 1.学的好的同学，就说自己可以独立完成接口自动化测试框架的搭建。

2.没什么信心的同学，就说自己可以套用其他项目的接口自动化测试框架来完成。

毕竟工作是开卷的，可以翻笔记资料，还可以百度。

*接口测试熟悉吗(接口自动化熟悉吗)?

1.还行，我会用 jmeter 执行接口测试。

先建立一个线程组，再添加 http 请求，填写 ip、端口、请求路径、请求方式及请求参数，并添加查看结果树。调通接口后，再设置参数化，断言等。

2.我也会 python+requests 接口自动化。吹一波“python+requests 接口自动化测试框架如何搭建”。

PS : 学的好的同学，把第 2 点的 python+requests 接口自动化说一下。

jmeter 测试环境怎么搭建的?

1) 安装 JDK，配置 JAVA 环境变量(JAVA_HOME、CLASSPATH、PATH)；

2) 解压 jmeter，配置 jmeter 的环境变量(JMETER_HOME、CLASSPATH、PATH)。

敲 jmeter.bat 命令或者双击它都可以正常启动 jmeter。

*python+requests 接口自动化测试框架如何搭建

我们使用 python+requests+pytest+allure/python+requests+unittest+ddt 进行接口自动化测试框架开发：

1. 首先，我们要构建一个测试工程，进行代码分层设计(核心层：api 层 + testcase 层)：

api 接口层：定义 api 接口类，类中存放访问接口的方法。

testcase 测试用例层：定义测试类，类中存放测试接口的方法。还可以引入参数化、断言、异常处理、日志处理。

common 通用层：定义公共方法，比如获取已登录的会话对象或者 token 值。

data 数据层：基于代码和数据分离的原则，参数化的数据文件存放此处。（.yaml 文件或 .json 文件）

log 日志层：日志文件的存放目录。

report 测试报告层：html 测试报告存放目录。

tools 层：存放一些自定义工具类，比如 DB 数据库类，类中定义访问数据库的方法，Log 日志类，类中定义日志的处理方式，还可以定义处理文件的类。

main_test.py 主程序：执行测试用例并输出 html 测试报告。

2. 脚本调试好，没有问题了，可使用 jenkins 进行持续集成的接口自动化测试，定时跑测试脚本，生成 html

测试报告, 并自动发送邮件给相关人。

PS: 最终写好的代码, 投入使用的代码, 一般不能出现 `print()` 语句, 需要通过类似 `logger.info()` 这种来代替。

*自动化测试框架是你自己搭建的吗?

刚开始是我们测试组员一起搭建的, 后面我也就能单独写了。

不懂还可以百度, b 站也有教程, 也有模板可以套用, 这个不难的。

我给你介绍一下搭建流程吧.....

*接口测试什么时候开始做

1. 从需求分析开始就做了。

2. 开发在需求评审后, 就会编写接口文档了, 然后评审接口文档, 评审后就要开始写接口用例了 (此时开发正在写接口代码)。接口用例评审后, 开发搭建测试环境, 就要进行接口测试了。

(接口测试流程和功能测试的流程基本同步。工作中我做接口测试, 其他同事做功能测试。

下一个版本, 组长可能就会互换任务, 方便让大家快速熟悉整个系统, 还能通过交叉测试发现新 bug)

PS: 1. 接口测试也叫灰盒测试, 开发在集成测试阶段进行接口调试。我们测试工程师实际安排在系统测试阶段进行接口测试。接口测试和功能测试并行。

2. 接口回归测试, 一般使用 `jmeter+ant` 或 `jenkins+ant+jmeter` 完成。

*你们做的项目是功能比较多还是接口呢? 功能占比是多少呢? 是每个接口都要测吗?

1. 功能测试和接口测试有时候一半一半的样子。有时候大家都分点接口来测, 有时候是我一个人专门负责接口测试。

2. 基本上每个接口都要测 (或者对重要模块的接口进行测试), 老模块的接口进行回归测试就行 (跑正例为主), 新模块的接口要一个一个测, 回归测试时新老模块接口都要测。

PS: 1. 实际工作中遇到接口测试的概率 1/3 左右。但为了提高面试通过率, 一定要吹牛自己接口测试很厉害。

2. 接口测试其实比功能测试更简单。并不是所有的项目都要做接口测试, 一般是比较复杂的项目需要做接口测试。

*接口自动化负责人数

一般项目组 5 个测试工程师, 很可能就 1 人负责接口测试及其接口自动化测试, 使用 `jmeter` 的概率会比较大。

PS: 1. 我们接触到接口测试的机会只有 1/3。工作中如果真的做接口测试, 其实会发现接口测试比其他同事在页面上点点点的功能测试更简单。

2. 也可能公司项目的接口不用测试工程师去测试, 开发自己测, 特别是简单的小项目。

3. 有些公司的项目也可能是每位测试工程师负责自己模块的接口测试及页面测试。

*物业项目的接口测试介绍

我做的物业客服系统, 主要使用对象为物业管理人员、小区客服、小区业主等。

主要功能有: 报事报修、报表统计、住户信息管理等。

我主要负责接口的功能测试, 整个系统 50 个接口左右, 在版本上线之后也测了部分常用接口的性能。

第一个版本的接口 30 个左右, 后期新增功能模块的接口 15 个左右。大概做了两期, 项目功能就基本稳定了。

PS: 物业接口的参数详见《物业管理系统 API 接口文档.docx》

*接口测试用例写了多少条

我们每个版本测试的新接口也不多, 15 个左右, 一个版本大概写 75-150 条接口测试用例吧, 平均一个接口写 5 条 10 条左右的用例。一条用例主要覆盖一个业务状态码。

PS: 1. 一个成熟的系统, 大概 100 个接口(100 张表)。

第一个版本的接口可能 30 个左右, 新增功能模块的接口可能 15 个左右。

2. jmeter 接口自动化测试用例数量(100 条左右, 执行 5-6 分钟) 我们把手写的接口测试的用例转为 jmeter 的 http 请求, 一个接口就是一个 http 请求, 一个线程组下可添加多个 http 请求。一个 http 请求有些会使用参数化来传值, 这样的话, 一个 http 请求就对应多个测试场景(多个测试用例)。一个线程组又相当于一个业务流程的测试用例了。且接口测试时分单接口测试、有依赖的单接口测试、业务流接口测试。所以具体的测试用例数量就不好算了。

3. python 接口自动化测试用例数量 (100 条左右, 执行 5-6 分钟) : 我们一个 test 测试方法就是一条测试用例, 并且测试一个接口, 有些会通过参数化考虑各种场景, 所以这种情况下的 test 测试用例其实颗粒度很大的, 包括了很多场景了, 细化一下, 其实是一条大的 test 测试用例包含了很多小的测试用例了。我们写的 test 测试方法还要分单接口测试、有依赖的单接口测试、业务流接口测试。所以具体的测试用例数量就不好算了。

(一般, 一个接口在 api 接口层的一个类中定义一个方法, 然后在 testcase 测试用例层的一个测试类中定义一个 test 测试方法去测试这个接口)

*如何编写接口测试用例

1. 拿到接口文档并且评审后, 我们一般使用 xmind 提取接口的测试点, 使用 excel 编写接口测试用例。

2. 输入参数主要使用等价类边界值来分析, 还要考虑必填和非必填项, 空和非空,

一个业务状态码就是一个测试点, 还要考虑接口的异常、接口的安全性。

3. 其实这和功能测试用例的设计没有什么区别。

PS: 主要考虑 5 大点, 1. 输入校验; 2. 输出检查; 3. 接口逻辑; 4. 异常处理; 5. 接口安全。

接口用例包含哪些内容?

模块名、用例名称、接口地址 URL、请求方式、请求头(headers)、请求参数(params)、请求主体(body)、取变量、预期结果。

接口测试用例的设计方法

和功能测试用例设计方法差不多, 主要有: 业务功能测试(场景法)、边界值分析法、参数组合测试、异常测试、安全测试。

*接口测试工具

1. jmeter、postman、Apifox、soupUI、fiddler。

2. jmeter 常用, postman 主要用于开发调试接口。

3. soupUI 用于 soap 协议(webbservice 技术)的接口测试。

*postman 用过吗

用过, 很简单, 他是谷歌浏览器 Chrome 一个插件(一般是单独的安装包), 安装和使用起来比较方便, 也是用来发送 http 请求的接口测试工具, 功能没有 jmeter 强大。开发工程师一般用 postman 调试接口, 我们测试工程师用 jmeter

比较多点。两者原理都一样，都是基于 http 协议发送请求得到响应，请求需要有请求地址、请求方式、请求参数。

PS：有些小公司可能会使用 postman 多点。

*jenkins 是什么

1.jenkins 是一款持续集成的工具，可实现脚本的定时运行、html 测试报告展示、邮件发送等等。

2.Jenkins 只是一个平台，真正运作的都是插件。这就是 jenkins 流行的原因，因为 jenkins 什么插件都有。

3.Jenkins 是基于 Java 开发的一种持续集成工具，用于执行重复的工作。(类似于流水线)

注意：1.一个项目多个版本情况下，可以在 jenkins 上创建多个任务，一个任务跑一个版本。

2.jenkins 的任务还可以定时拉取 git/svn 远程仓库中的测试脚本，用于定时执行。

3.开发或运维用 jenkins 定时从 git/svn 远程仓库拉取项目的最新代码用于编译、打包、部署（自动发布 war 包到测试环境中的 webapps 目录下）。

4.jenkins 主要用于回归测试阶段。

*如何使用 jenkins+ant+jmeter 进行持续集成的批量接口自动化测试

1. 准备 jmx 接口测试脚本和 build.xml 构建文件（配置了构建任务名、jmeter 安装根目录、jmx 测试脚本目录、测试报告生成路径）

2. 通过 jenkins 新建任务，构建触发器、构建 ant 执行命令；

3. jenkins 定时执行时，生成测试报告、并自动发邮件。

PS：1. ant 是一个将 java 项目进行编译、测试、部署的一个自动化构建工具。

2. ant 在执行时，会自动读取当前目录下的 build.xml 文件。

3. ant 调用 jmeter 来运行 jmx 脚本，生成 jtl (xml) 格式的测试结果，之后将 jtl 文件转为漂亮的 html 测试报告。

4. jmeter+ant 主要用于接口测试的回归测试阶段生成 html 测试报告。

*如何使用 jenkins+python+requests 进行持续集成的接口自动化测试

1. 准备 python+requests 接口自动化测试脚本；

2. 新建 jenkins 定时任务，构建触发器、构建 python 执行命令；

3. jenkins 定时执行时候，生成测试报告、并自动发邮件。

PS：jenkins 的定时执行一般都是用于回归测试阶段。

*接口交互的数据格式有哪些(请求或响应的数据格式)？

JSON、XML、HTML 文本，常用的是 JSON 格式的文本。

*请求体如果是 json 格式的数据，该怎么设置？

在请求头设置内容类型 content-type：application/json，且必须使用 post 方式。

（python+requests 在 post 方法中使用 json 参数名）

*接口支持的请求头内容类型 content-type 有哪些？

1. application/x-www-form-urlencoded; charset=utf-8：表单默认以此方式提交普通文本数据，后面的 charset=utf-8 是默认的字符编码。支持 get、post 方式。

（username=13000000000&password=123456）

2. multipart/form-data：表单用于上传文件或提交文本数据，兼容第一种类型。仅支持 post 方式。

3. application/json : 传递到后台的是序列化后的 json 字符串。仅支持 post 方式。
4. text/xml : 请求数据格式是 xml , 可以用于 Webservice 技术的 soap 协议 , 仅支持 post 方式。

token 和 sessionid 的区别

简单区别 :

- 1.token 和 sessionid 都是用于登陆后的身份验证的, 因为 http 协议本身没有状态。
- 2.token 是服务器生成的一个字符串, 比较轻量级, 常作为请求头参数 authorization 的值。
sessionid 是服务器创建会话对象而具有的唯一属性, 作为 cookie 存放到浏览器。
token 越来越流行, 用于取代 sessionid, 因为服务器端的 session 会话对象比较重量级。

补充区别 :

- 1.传统的前后端合在一起的 web 项目使用 cookie+sessionid 技术维护用户的登录状态。服务器会为每个客户端创建一个 session 会话对象, 重量级一点。
- 2.前后端分离的项目, 通过 token 机制来控制用户的登录状态。token 常作为请求头参数 authorization 的值。
token 值主要用于用户权限验证, 代替 jsessionid 的作用。服务器一般使用 redis 缓存数据库保存用户的 token , 用于验证客户端请求携带的 token。token 本身是字符串, 比较轻量级一点。
(token 和 sessionid 都是用于验证登录身份的, 任何需要验证登录权限的接口都需要依赖登录接口)

PS :

- 1.python+requests 中处理 sessionid 可以使用本地的 session 对象发送请求, 同一个 session 对象, 共享同一套 cookie(sessionid)。
- 2.jmeter 中可以在某个线程组下添加 cookie 管理器, 让该线程组下的 http 请求共享同一套 cookie(sessionid)。
- 3.php 项目的 sessionid 一般叫 phpsession, java 项目的 sessionid 一般叫 jsessionid。
- 4.jwt Token 技术比 session 技术更优, 因为服务器要保存(维护)每个用户的 session 对象, 而服务器却不需要保存用户的 jwt Token。而普通的 token, 服务器一般需要保存到 redis 缓存数据库中。

*token 或 sessionid 的值怎么设置 ?

token 值一般登录提取后, 放到请求头 authorization 认证字段中。(jwt token 可能这样设 Bearer eyJhbGciOiJI6I)

sessionid 登录提取后, 以键值对格式放到请求头 cookie 字段中。

PS :

- 1.jmeter 工具需要添加 http 信息头管理器或者 cookie 管理器,
python+requests 需要在请求方法的参数列表设置 headers 参数或者使用本地 session 对象发请求。
2. token 的位置 : 在请求头参数 Authorization (比较常见), 也可能是请求头参数名 token 记录着, 也可能是作为请求体参数。
3. 接口的鉴权怎么测 : 接口的鉴权主要通过 token 来控制, 需要考虑 token 的有效性、时效性。

*jmeter 和 Python 测接口, 如何设置 cookie ?

- 1.如果是 jmeter 的话, 可以添加一个 cookie 管理器。

2.如果是 python+requests 的话,可以使用本地 session 会话对象发送请求。

PS : 1.如果在 jmeter 手动添加 cookie 的话,是在信息头管理器设置头信息 cookie 或者再 cookie 管理器中添加。

2.如果在 requests.get()方法或 post()方法,可在参数列表设置 cookies 参数 (或设置 headers 参数)

***接口测试使用的协议或你们 https(SSL)协议的接口怎么测试?**

测试环境中我们一般使用 http 协议进行接口测试 (或接口的性能测试),等上线时开发(或运维)才会设置为 https。

PS : 1. jmeter 进行 https 协议的接口测试或接口性能测试时,可以在 jmeter 中手动添加 ca 安全证书。

***一个接口的请求参数需要依赖另一个接口的返回数据, Jmeter 和 Python 分别怎么做?**

假设有两个接口, A1 和 A2, A2 接口的请求数据依赖 A1 接口响应的数据。

jmeter : 在 A1 接口中添加边界提取器 (或正则表达式提取器或 json 提取器),用变量名记录获取到的目标数据,然后在 A2 接口中使用 \${变量名} 的方式进行引用。

Python : 通过 response.text 属性获取响应体数据,然后使用 str.split()切割方法高仿“边界提取器”,针对左右边界字符串切两次,即可获取到目标数据了,可传给访问 A2 接口的方法。

或者通过 response.json()方法高仿“json 提取器”,把 json 字符串转为字典后,根据键来获取值 dict[key]

***接口如何关联 (接口之间有依赖怎么处理、接口之间的参数传递)**

同上。

***接口如何交互 (接口先后调用的逻辑)**

思路: 页面一个功能请求往往会涉及 2-3 个接口。比如登录接口(联塑管道项目),需要预先调用“aes 密码加密接口”和“获取 tokenkey 接口”。

答: 1.商城项目: 比如在页面调用下单接口: 需先调用登录接口,再调用查看商品详情的接口,再调用添加购物车接口,再调用结算的接口,最后调用下单接口。(做接口测试,其实还可简化过程: 先调用登录接口提取 token 值,再调用下单接口即可,把下单接口需要的请求参设置好即可)

2.借贷项目: 比如在页面调用申请借款接口: 需先调用登录接口,再调用检查订单接口,最后调用申请借款接口。(做接口测试,其实还可简化过程: 先调用登录接口提取 token 值,再调用申请借款接口即可,把申请借款接口需要的请求参数设置好即可)

PS : 工作中,问开发 或 通过“fiddler 抓包”或“jmeter 代理服务器录制脚本”,来了解接口先后调用的逻辑。

***接口 mock 了解嘛 (一个接口依赖第三方接口,但是第三方接口暂时无法调通,你们怎么办?)**

开发会写第三方模拟接口(mock 模拟接口)作为挡板,给自己系统暂时调用。不做实际业务处理,单纯返回相应的结果。

PS : 第三方接口默认没有 bug,一般不需要单独测试,开发自己调试好就行。

在测试环境中,开发可能写 mock 接口来代替第三方接口。

***jmeter 的提取器有哪些?**

常见的有边界提取器、json 提取器、正则表达式提取器。(还有 xpath 提取器,这些都属于后置处理器)

PS : 边界提取器是 jmeter 5.0 版本才开始有的新功能。

*jmeter 请求默认值用来干嘛

1. http 请求会继承“默认的设置”，比如继承 ip、端口号等。如果换新测试环境，只需要在“请求默认值”处修改为新的 ip 即可。（用于统一 ip 和端口号，一变则百变）
2. “请求默认值”还可以设置代理服务器，这样所有的请求都可以使用此代理服务器。

*jmeter 用户定义的变量用来干嘛

用于多处请求设置同一个值时，对某一个参数值的多次引用。（一次定义多次调用 或 一处定义多处调用）

*jmeter 如何设置全局变量而跨线程组来调用

场景：比如从登录提取的 token 值需要用于多个线程组下的 http 请求，

此时则需要把提取的 token 值设置为全局变量而被其他线程组调用。

答：很简单，首先添加“BeanShell 取样器”把提取的 token 值通过 `$_setProperty(newtoken,${mytoken},)` 设置为全局变量，然后在其他线程组下使用 `$_P(newtoken,)` 来调用全局变量名记录的值。

- PS：1. 或者使用“BeanShell 后置处理器”，同理。（均需提前在“测试计划”界面设置“独立运行每个线程组”）
2. 后置处理器是在 http 请求后执行的，前置处理器是在 http 请求前执行的。
 3. 一个线程组相当于一个测试方法 test_xx()。
 4. 具体的设置，请参考 <https://www.cnblogs.com/crowbrother/p/16164735.html>

逻辑控制器用过吗？

1. 用的比较少，主要有“如果(if)控制器”、“循环控制器”。
 2. “如果(if)控制器”是指满足指定条件下才执行下一个 http 请求，例如，如果登录接口响应的 token 不为空，就执行接下来的接口。否则，不执行接下来的接口。循环控制器，就是控制其中的 http 请求的循环执行次数。
- PS：jmeter 配置元件有哪些：CSV 数据文件、信息头管理器、Cookie 管理器、请求默认值、用户定义的变量等。

如何保证一个接口不出问题才调用下一个接口

使用“如果(if)控制器”

接口的状态码有哪些？和服务器响应的状态码有什么区别？

1. 接口响应的业务数据不一样，返回的状态码也不一样，接口的状态码是由开发统一定义的，比如，我们的物业项目的获取账号信息接口，成功的状态码是 0, 40017 表示此 token 不存在，40011 表示无此用户，还有一些其他的状态码。
2. 服务器响应的状态主要有 200 ok、404 not found、500 服务器内部错误。存放到响应头(响应行)中。

PS：1. 只要客户端能正常接收到业务数据(任意的业务数据)，那么，服务器响应的状态码是 200。

2. 状态码（响应码）分为“业务状态码”、“响应状态码”。前者是开发者自定义的，后者是服务器定义的。

*http 协议接口常见的方法有哪些（提交表单的方式），有什么区别

主要有 get（查）、post（增）、put（改）、delete（删）等等。

实际开发中, 一般只用 get(查)、post(增删改), 具体如何使用是开发者自定义的, 不是强约束。

*如何判断一个 bug 是前端还是后端问题

1. 可借助 F12 或者 fiddler 抓包查看请求响应, 进行辅助分析。
2. 请求 url 及请求参数的问题, 或响应状态码 4 开头的问题, 一般就是前端问题。
3. 业务逻辑的问题、响应数据内容的问题, 或响应状态码 5 开头的问题, 一般就是后端问题。
(后端问题一般还需要查看日志中 Exception 异常信息, 开发可了解到第几行代码抛出了异常)

PS : 1. 当状态码是 200, 响应体的内容也可能是错误的, 内容的正确与否, 由人来判断。

2. 发现 bug 后, 排查问题的顺序是: 看页面, 抓包看请求响应, 查看 linux 日志、查看库表。

*接口重要数据是怎么加密的 (重点)

1. 金额、数量等敏感数据 : 拼接规则是 “敏感数据+用户名+时间戳+密钥”, 使用 md5 或 sha2 加密算法生成密文。
发送两份数据 (明文+密文 sign) 给接口。接口收到明文后, 按照一样的拼接规则 (结合服务器保存的密钥) 生成新密文并比对, 一致则表明数据没有被篡改过。我们可使用 fiddler 抓包后, 打断点修改请求的金额、数量等数据, 看后台接口怎么反应。

2. 我们之前 “接口的鉴权” 主要通过 jwt Token 来控制接口的访问权限 和 有效期。

jwt 分为三部分: header 头部、payload 载荷、sign 签名。前面两部分数据相当于 “裸奔” (base64 编码), 服务器接口接到 jwt 后, 会把前面两部分字符串 xxx.yyy 加入同一个密钥用 sha2 算法生成新的 sign 签名并比对, 一致则表明 jwt 没有被修改过, 最后才读取 payload 载荷里面的用户名和有效期等关键信息。

(登录成功后服务器才会响应 jwt Token 给客户端, 客户端下次访问时携带 jwt 即可。jwt Token 比较轻量级, 并且服务器不需要保存。)

PS : 1. 加密算法分为 “不可还原” 和 “可还原” :

不可还原: md5 或 sha2 等哈希算法生成的密文是不可被还原的 (所谓的还原是使用穷举法进行暴力破解)。

可还原: aes 对称加密算法生成的密文可以被还原, 只要加密和解密的密钥是同一个即可。

rsa 非对称加密算法生成的密文也可以被还原, 使用公钥来加密, 解密时使用成对的私钥即可。

https 协议的加密, 主要使用的就是 aes 对称加密或 rsa 非对称加密算法(更安全点)。

gitee 网站的 ssh 公钥登录, 使用 rsa 加密算法来进行身份认证。

2. token 有很多种, 常见的是 jwt (json web token) 的 token ,

jwt Token 的详细介绍: <https://www.cnblogs.com/crowbrother/p/14813754.html>

3. 前后端分离系统的登录接口: 一般是通过输入的用户名和密码, 密码在前端服务器加密之后再从用户端传给后端接口(以免传输中密码被窃取了), 后端接口去库表中查询用户名密码是否存在。

4. 安全是相对的, 没有绝对的安全, 能控制时间的有效性越短则越安全、敏感数据传输的次数越少则越安全。
系统的安全性测试, 可以从接口的安全性方面说。安全测试也叫渗透性测试。

aes 加密密文如何生成 或 sign 签名如何生成?

很简单, 在本地调用写好的 python 方法即可生成 (我们之前写过这个 aes 加密方法、获取 sign 签名的方法, 可以直接拿来调用的)。 (bejson 网站也有工具生成)

*接口的安全性测过吗, 或系统的安全性怎么测

同上

*接口测试的关注点？

- 1、接口的功能：关注响应头的状态码 200、404、500 等，响应体数据的内容(包括业务状态码)与内容格式。
- 2、接口的性能：关注接口响应时间，2-5-8 秒原则。
- 3、接口的安全性：接口的调用一般都有访问权限的控制（鉴权），不能随意调用，一般通过 token 控制访问的权限。
(传统项目的接口鉴权通过 sessionid)

*怎么验证接口是通过的？

- 1.主要检查服务器响应给客户端的信息(响应报文)和预期是否一致。(或使用断言来自动判断实际结果和预期是否符合)
 - 2.再顺便检查一下请求数据(请求报文)是否正确。
- 补充：1. 还要查看库表，检查接口是否正常落库了某些数据。
2. 接口的响应时间是否符合需求，基于 2-5-8 秒原则。

*接口出错了，怎么分析处理？

- 1.先看响应头信息的响应码，404 检查一下请求路径 url 是否错误、400 检查请求路径 url 和请求参数的格式是否正确、500 检查请求参数的内容问题，修改后再进行测试。
- 2.如果没有任何响应，报错"连接超时"，则 ping 一下服务器 ip 并 "telnet <ip> 8080" 检查 tomcat 端口是否开启。

接口自动化做了多久

每个版本都会做接口测试，开发写好接口，我拿到接口文档之后，就开始做着，回归测试阶段还要回归老模块接口。

接口自动化的覆盖率（挖坑题）

接口自动化的覆盖率为 100%，所有接口都要测，所有状态码都要覆盖到位。

(功能测试用例的覆盖率也是要求 100%，要求用例覆盖所有需求场景)

*接口有哪些参数？

- 1.接口发出请求时，有请求头参数和请求体参数；接口响应时，有响应头参数和响应体参数。
- 2.比如，登录接口的请求参数一般是：username、password。然后会响应 token 值。
下次发出请求时携带该 token 值用于身份验证。

PS：有些接口请求需要依赖特定的请求头信息，需要我们先设置好。（头信息是可以自定义新字段的）

*接口测试遇到过什么问题

思路：由于什么操作而出现问题，后来又是怎么分析处理的。

- 1、常遇到的是接口文档的问题，因为开发是先写好接口文档再编写代码，接口文档的参数有时和实现的不一致，开发又忘记修改接口文档，然后造成做接口测试的时候请求不通过，只能去找开发确认，耽误测试时间；
- 2、刚开始，后端开发不喜欢验证接口参数，接口不能过滤掉一些非法数据，有时连必填项都不进行校验。
后面开会讨论，不仅前端要对非法数据进行拦截，后端也要对非法数据进行校验。
- 3、有时我们提交的接口 bug，后端开发不认为是 bug，理由是应该由前端做处理。比如商品查询接口，后端直接返回所有商品数据让前端去做分页，这明显不合理，后端应该做好分页处理来减少数据量的传输。

*接口测试发现了什么 bug

通用 bug：

- 1.越权问题：用户登录后获取 token，然后当前用户可以使用自己的 token 修改他人的昵称，存在越权的问题，之后发现所有的接口都存在此严重问题。
- 2.用户注销登录，原 token 还能继续使用：提 bug 单后，后端开发不愿意改，说这个在前端已经处理（前端会删除原 token），用户在使用不会出现问题。我认为这是一个很严重的安全问题，说服不了开发后，拉着 BA 产品经理、测试组长、开发组长一起开会讨论后，开发才愿意修改此 bug。（注销登录时：前端开发会删除 token，后端开发应该在 redis 中保存该用户名及对应的注销状态码，并设置一定的期限）

项目 bug：

- 1.借贷项目：管理员审核接口，当设置的审核状态码非法时，也能落入数据库表。原因是开发没有对审核状态码进行有效性校验。（借款金额为 0 或负数也能成功，因为接口没有校验个人的可借款额度的范围）
- 2.电商项目：添加新商品这个接口，当设置的库存量的值为空时，接口报错 500。原因是开发没有对空值进行处理，应该自动给默认值 0。
- 3.物业项目：登录接口，删除状态的无效账号也能正常登录且获取到 token。原因是开发没有对账号的状态进行校验。查询工单接口，修改时间字段显示的是不能直接看懂的时间戳。原因是开发没有对时间戳进行转换处理。



PS：接口测试发现的 bug 一般属于后端 bug，相当于功能测试在页面上点点点发现的“后端 bug”，这与业务逻辑有关，响应的数据本身有问题。

*跑自动化代码遇到过什么问题

报错抛异常：日志文件路径报错、请求地址报错、请求参数格式报错、响应结果转字典报错（resp.json()方法）

*接口有测过什么异常场景吗

1. 业务功能的异常场景，一般是根据“业务状态码”去测（反例状态码）。
2. 比如，登录接口的异常场景有：用户名或密码格式有误，用户名或密码错误，用户不存在，用户已注销。用户已登录，登录超时等等。

下单接口的异常场景有：用户未登录，token 已过期，商品库存不足，商品已下架，地址有误，下单超时等等。

PS：业务状态码可以现编，比如，40010-用户未登录、40001-token 已过期、0-success 成功等等。

*除了接口超时外，还有哪些异常场景

除了业务功能的异常场景, 还有请求路径有误 (404)、请求参数有误、权限不足 (403)、未认证 (401)、服务器超载 (503)、服务器内部错误 (500) 等等。

python 做接口测试用到哪些库(模块)、哪些方法

- 1.用了 requests、pytest、allure-pytest、yaml、os、unittest、ddt 、HTMLTestRunner 库 ;
- 2.requests 用来发送请求 (常用的方法就是 post , get 来发请求) , pytest 用来编写用例和做断言 , yaml 文件用来做参数化 , allure-pytest 用于生成测试报告。
unittest 用来编写用例和做断言 , ddt 用来做参数化 , HTMLTestRunner 用于生成测试报告。

PS : 模块、库、框架是同义词, 比如 pytest 就是一个框架, 也是一个模块, 也是一个库。

python 做接口测试时, 测试数据如何维护 ?

- 1.如果是 unittest 模块 : 可以写在测试方法上, 也可以写在 json 文件中。数据多则写在 json 文件里, 使用 @ddt.file_data()和@ddt.unpack()数据驱动的方式来引用数据。
- 2.如果是 pytest 模块 : 可以写在测试方法上, 也可以写在 yaml 文件。数据多则写在 yaml 文件里, 读取 yaml 文件。

PS : 如果线上环境跑自动化, 测试数据如何切换 ?

答 : 多准备一份测试脚本 (拷贝一下, 简单修改一下) , 里面的 data 层存放线上自动化的参数化数据即可。

jmeter 做接口测试时, 测试数据如何维护 ?

- 1.通过 csv 数据文件, 把测试数据维护起来 (保存起来) 。

*python 做接口测试时, 参数化怎么做 ?

- 1.如果是针对 unittest 模块。先在继承了 unittest.TestCase 的测试类上声明装饰器@ddt.ddt。
- 2.然后在测试方法上使用装饰器@ddt.data({字典}) 或 @ddt.file_data('json 文件路径')进行参数化 , 还要绑定@ddt.unpack()用于解析数据, 通过键值对的键把值传给方法的参数名。

*jmeter 做接口测试时, 参数化怎么做 ?

添加 “CSV 数据文件设置” 后, 引入数据文件, 通过自定义的变量名来逐行取值\${变量名} 。 (或通过 CSVRead 函数)

*python+requests 接口自动化测试的常用方法, 并介绍一下常见的方法参数

有 session.get() 和 session.post() 方法 ,

参数名 params 记录的参数值自动放在地址 url 后面 ,

参数名 data 记录的参数值自动以普通格式存放到请求体 body 中 ,

参数名 json 记录的参数值自动以 json 字符串格式存放到请求体 body 中, 并自动添加内容类型 application/json ,

参数名 url 记录的是访问接口的地址, 参数名 headers 记录的是头信息。

PS : 1.以上参数值在 python 中都是字典类型。get()方法使用 get 方式发出请求, 仅适用 params 参数名。

2.参数名 proxies 存放代理服务器的地址。

3.还有 requests.get()、requests.post()。session 对象发请求会共享同一套 cookie , 共享同一个 sessionId。

*jmeter 和 python , 断言怎样做 ?

jmeter 断言 :

- 1.针对目标请求, 添加响应断言, 填入需要断言的内容 (期望结果) ;

2.如果这个接口还做了参数化(有多次请求)，且每次请求得到的响应内容不一样。

则可把断言的“内容”也进行参数化，需要先在 CSV 文件中添加一行“预期结果”，然后在响应断言中使用 `${expect}` 来引入“期望值”。

python 的断言：

python 可用自带的 `assert` 关键字来做断言的，也可用继承父类 `unittest.TestCase` 的 `self.assertEqual()`、`self.assertIn()`断言方法。

(如果有参数化，针对不同的响应结果，可把断言的“期望值”进行参数化)

PS：断言的内容一般是字符串，断言的结果是 `bool` 布尔型值：`True` 或 `False`。

***做过哪些接口的测试？举例说一下你的接口测试是怎么做的？**

1. 每个版本新增的功能接口要进行测试，回归测试时候，老版本的所有接口也要进行回归测试。

比如“**借贷系统**”：注册接口、登录接口、个人信息接口、申请借款接口、后台审核接口、还款接口等等。

比如“**电商系统**”：注册接口、登录接口、添加到购物车接口、购物车勾选商品接口、购物车结算接口、提交订单接口（取消订单接口）、支付接口、评价接口等等。

比如“**物业系统**”：登录接口、获取账号信息接口、查询用户所在小区接口、获取所有小区列表接口、查询房屋详情接口、查询工单列表接口、添加工单接口等等。

2. 我的接口测试是这么测的：

1. 当我们用 `jmeter` 执行接口测试时：

先建立一个线程组，再添加 `http` 请求，填写请求 `ip`、端口、请求方式、请求路径、请求参数，并添加查看结果树。调通接口后，还可以设置参数化，断言等。

2. 当我们用 `python+requests` 进行接口自动化测试时：

我们选用 `python+requests+pytest+allure` 技术进行接口自动化测试框架的开发：

2.1 首先，我们要构建一个测试工程，进行代码分层设计(核心层：`api` 层 + `testcase` 层)：

`api` 接口层：定义 `api` 接口类，类中存放访问接口的方法。

`testcase` 测试用例层：定义测试类，类中存放测试接口的方法。还可以引入参数化、断言、异常处理、日志处理。

`common` 通用层：定义公共方法，比如获取已登录的会话对象或者 `token` 值。

`data` 数据层：基于代码和数据分离的原则，参数化的数据文件存放此处。（`.yaml` 文件/`json` 文件）

`log` 日志层：日志文件的存放目录。

`report` 测试报告层：`html` 测试报告存放目录。

`tools` 层：存放一些自定义工具类，比如 `DB` 数据库类，类中定义访问数据库的方法，`Log` 日志类，类中定义日志的处理方式，还可以定义处理文件的类。

`main_test.py` 主程序：执行测试用例并输出 `html` 测试报告。

2.2 脚本调试好，没有问题了，可使用 `jenkins` 进行持续集成的接口自动化测试，定时执行测试脚本，生成 `html` 测试报告，并自动发送邮件给相关人。

*举例说明“下单接口”怎么测？

1.下单这个接口用的是 http 协议，使用 post 请求方式，发送给服务器的参数有 token、产品 ID、sku 商品规格、购买数量，收货人地址编号等等，这些参数都是必传的参数。

2.我们是使用 Jmeter 来做接口测试的，首先，要新建一个线程组，在线程组下面添加一个 http 的请求，然后填写好服务器 ip 地址，请求方式，请求路径，请求参数。

因为 token 值是用来验证用户合法身份的并且是动态变化的，并且这个值是从登陆接口中响应过来的，所以，我们要在下单这个接口的前面添加一个登陆的接口。

在登陆接口请求中使用边界提取器把 token 值提取出来，然后在下单接口中使用\${参数名}的方式引用。

(接下来还可添加参数化，构造各种正常和异常的数据，我们先在本地创建一个 txt 文档，

把参数填写到文档里面，在 Jmeter 中添加一个 csv 文件，填写好 txt 文档的路径，然后在请求参数中使用 \${变量名}的方式引用；

接下来还可添加断言，用于自动判断响应结果和预期结果是否一致。(断言关键字一般是业务状态码及描述信息)如果断言的内容(预期结果)会根据请求参数的变化而变化，则断言的内容也要参数化。

最后，添加“查看结果树”。)

*商城项目的接口及参数

1.添加购物车的接口：token、产品 ID、价格、规格、数量等等。(token 用于验证身份，可关联到用户编号)

2.购物车结算的接口：token、产品 ID、价格、规格、数量等等。(后端接口会自动查询优惠信息进行优惠扣减)

3.下单接口：token、产品 ID、规格、数量、地址编号、优惠券编号等等。

4.申请支付的接口：token、订单编号、支付方式等等。

5.提交支付的接口：token、订单编号、支付密码等等。

*借贷项目的接口及参数

1.申请贷款的接口：token、金额 money、期数 month 等等。

2.提交订单的接口：token、订单编号 ordernum 等等。

3.还款的接口：token、(某期) 还款编号等等。

4.提交支付的接口：token、还款订单编号、支付密码等等。

PS：1.一次页面点击，往往会先后执行 2-3 个接口。也就是页面上的一个基本功能大概会涉及 2-3 个接口。

2.你们借贷系统还有那些接口：授信提交接口、资料保存接口、资料提交接口、冻结解冻接口等。

*物业项目的接口及参数

1.登录接口：用户名 username、密码 password。

2.获取用户所在小区列表接口：token、用户名 username。(token 作为请求头参数 Authorization 的值)

3.查询用户楼栋房屋信息列表接口：token、用户名 username。

4.获取所有工单列表接口：token、小区编号 communityId、页面数量 pageNum、页面大小 pageSize。

*举例说明“借款接口”怎么测？

1.先调用“登录接口”获取 token。可设置“请求默认值”用于统一 ip 地址，可设置“用户定义的变量”方便修改值。

- 2.再调用“检查订单接口”，传入 token，校验该用户是否存在逾期订单、未还清的订单或其他的问题订单。
- 3.再调用“申请贷款接口”，传入 token，传入金额和期数。
- 4.再调用“确认订单接口”，再次传入 token，传入金额和期数。
- 5.再调用“提交订单接口”，传入 token，传入上个接口返回的“订单编号”。

后面就是调用管理员的审核接口和放款接口了。(金融等敏感数据一般都会先加密,发送两份数据:明文+密文 sign)

*举例说明“还款接口”怎么测？

- 1.先调用“登录接口”获取 token。可设置“请求默认值”用于统一 ip 地址，可设置“用户定义的变量”方便修改值。
 - 2.再调用“还款接口”，传入 token，传入还款金额（包括本金和利息）和期数（第几期）。
- (金融等敏感数据一般都会先加密,发送两份数据:明文+密文 sign)

支付功能，付款的接口是怎么测试的呢？

第三方的支付接口、充值接口、提现接口默认没有 bug，一般是由开发直接调试成功即可的，不需要我们去进行接口测试。

比如，像支付宝、微信这些第三方支付，开发会直接调试好。

支付功能，如何确保安全性，防止用户“薅羊毛”？

用户输入的密码和金额等敏感数据在客户端进行 md5 或 sha2 加密，加密前的拼接规则自定义：

比如，“敏感数据+用户名+时间戳(服务器响应的时间戳)+固定数字”。加密后同时发送明文和密文数据给服务器，如果数据被黑客拦截并修改了敏感数据，那么服务器根据已知规则加密明文后发现与密文不一致，则拒绝此请求。

如果快速注册 1000 个账号

- 1.使用 python+selenium 技术循环执行 1000 次注册的方法；
- 2.使用 jmeter 或 python+requests 循环向注册接口发送 1000 次请求即可；
- 3.使用存储过程循环插入 1000 条用户数据。

PS：如何准备 1000 个用户名数据：可以利用代码或者 excel 表格生成，然后放到参数化文件中使用。

*如果让你一个人负责接口自动化，可以吗

1. 没问题的，jmeter 和 python+requests 都用的比较多，我单独负责过 xx 商城（xx 借贷、xx 物业）的接口测试，写过接口测试用例，也写过 python 接口测试脚本和创建 jmeter 接口测试脚本，熟悉代码的分层设计。
2. 这些操作我在公司都有整理过操作文档，也有现成的接口自动化测试框架可以拿到套用。

(不懂的还可以百度)

*接口测试与功能测试的区别

1. 接口测试属于集成测试阶段，接口测试能更早更底层的发现 bug，降低修复成本。(我们实际在系统测试阶段测接口)
2. 接口测试关注的是数据(请求路径、请求报文、响应报文)。只要数据正确了，功能也就基本没有问题了，剩下的就是前端开发如何把这些数据展示在页面上
3. 功能测试属于系统测试阶段或用户验收阶段，属于黑盒测试。关注的是页面功能是否能正常使用。
4. 接口测试和功能测试其实都是关注系统的功能，前者是更早地更底层地测试，后者是站在用户角度去测试。

PS：我们在页面上点点点的顺序，就是接口调用的先后顺序，两者是一致的。多接口的调用是讲究顺序的。

功能测试的目的

发现系统的 bug，提升软件的质量。

*为什么要做接口测试或接口测试的目的与意义（进行接口测试的优势）

1. 尽早介入测试，可以提前发现 bug，更底层发现 bug，降低企业的修复成本。
2. 接口测试更易实现自动化持续集成，也比 UI 自动化更加稳定点。（因为页面可能会经常改变）
3. 绕过前端太容易了，只依赖前端进行限制，已不能满足系统的安全要求了。

前后端传输的敏感信息(金额、银行卡号、密码等)也需要抓包检查是否加密。

（现在很多系统的前后端架构是分离的，分别部署在不同的 web 服务器上，比如小程序。）

PS：比如在页面上输入用户名或金额，一般前端 js 会剪掉头尾可能存在的空格。

那么，接口是否也会重复剪掉头尾空格呢，则需要用到接口测试工具来测试。

*接口是一个一个做测试，还是系统性地做？

接口测试主要分三步：第 1 步测试单接口；第 2 步测试有依赖的单接口；第 3 步测试业务流接口。

PS：1. 业务流接口主要考虑正例。一个线程组对应一个业务流程，对应一个测试方法。

2. 单接口测试 和 业务流（场景）接口测试的区别：后者需要考虑接口之间的交互顺序 和 数据流转（提取器）。

你们脚本或接口测试脚本如何管理(自动化脚本如何维护)

.py 或 jmx 接口自动化脚本使用 svn 或者 git 来进行版本管理。

PS：py 脚本，同事间一般各写各的，执行时候分别执行，或者使用 jenkins 关联在一起先后执行。

*app 接口测试和 web 端接口测试有什么区别？

1. app 端接口（包括小程序、H5）和 web 端接口都是服务器的接口，对于测试来说基本没有区别。
2. app 端接口一般使用 api token 和 user token 双 token 来控制访问权限，前者控制当前 app 访问接口的权限，后者控制当前用户访问接口的权限。

一个线程组设置几个场景？

一个线程组相当于一个 test 测试用例方法，所以一个线程组一般设置一个场景。也就是一个线程组是一条流程性的测试用例。

PS：1. 一个线程组下可添加多个 http 请求。

2. 接口测试时分单接口测试、有依赖的单接口测试、业务流接口测试。

*接口自动化常识

1. 接口脚本：jmeter 的脚本文件.jmx，python 接口自动化脚本.py 文件。
脚本的目的是为了完成一个工作流程，程序的目的是为了完成某个功能。
2. 接口测试也叫后端测试（后台测试、服务器测试）。API：Application Program Interface 接口。
3. 登录接口有多少个，淘宝后端登录接口有 3 个(接口分 3 套)，分别对应 web 端、安卓 app 端、苹果 app 端。
一般公司的项目可能各个前端都是使用同一套接口，也就是登录接口就一个。
4. 自动化脚本往往一个版本一套脚本。每次新版本都在原来基础上添加新的脚本。老版本的脚本也要备份一份。

5.jmeter 和 python+request 都属于接口自动化技术。

6.公司如果问，给 30 个接口，要多久全部搞定，回答 1-2 周。然后 python+request、jenkins 自动化运行。

一条接口自动化用例，执行时间大概 1 秒内，整套接口(50-100 个接口)执行时间 3-5 分钟左右。

7.python+request 框架在工作中一般不需要从零开始搭建，有现成的其他项目的测试框架拿来套用。

(或者拿章老师教的框架来使用)

8.服务器接口是用 java 语言编写的，前端页面是用 H5 编写的。ios 的 app 是用 swift(object-c)语言写的，

安卓的 app 是用 java、kotlin 编写的。

***web(UI)自动化测试做过吗**

(可以拒绝回答)

1.我们之前做的系统前端页面不太稳定，变化比较快，所以就没有做 web 自动化。我做过 python 接口自动化。

2.但是我也会 python+selenium 的 web 自动化，比如八大元素定位，主要使用 id、xpath 定位方式。

测试框架的设计使用 po 模式来组织代码，主要分 page 页面层、business 业务层、testcase 测试用例层。

3.其实 python+requests 接口自动化才是我的强项，介绍一下代码分层。

PS：web(UI)自动化技术主要回归稳定的老模块。

Python+requests 也是用于回归稳定的老模块接口。

自动化技术都是用于回归测试(稳定的老模块)，用于减轻测试人员的负担。

自动化测试的意义(web 自动化)

1.释放双手，减少人力成本。可重复执行测试脚本，提高老模块回归测试的效率。(web 自动化不是为了测出 bug)

2.由于每次重复执行的代码具有一致性，减少人为的失误。

3.还可以使用 jenkins 定时执行，安排晚上的时间进行，更好地利用资源。

4.性能测试

*性能测试怎么做的 或 性能测试流程

比如,我这个 xxx 项目的性能测试是这样做的:(性能测试一般指接口性能测试,一般是系统稳定后再做)

1、先性能需求分析,挑选了用户使用最频繁的功能来做性能测试,编写性能测试用例。

(比如:挑选了登陆、打开系统首页、搜索、添加购物车、结算、提交订单接口。确定性能指标:事务通过率为 98%, 90%的事务响应时间不超过 3 秒,CPU、内存、IO 的使用率为 75%以下)

2、搭建性能测试的本地环境,服务器项目环境让开发协助搭建。准备好性能测试数据。

(性能测试数据,包括基础数据和使用数据,如果我们要测试系统半年内是否能稳定提供服务,就要在估算半年内,系统会有多少数据,然后在数据库上导入对应量的数据,这叫基础数据;使用数据,就是我们测试过程中需要用到的参数化的数据)

3、编写性能测试脚本(jmx),填写接口 url、请求参数,并优化,包括:参数化,断言,接口关联(边界提取器)等。

4、设计性能测试场景,我们这个项目做了单用户单功能循环 200 次的基准测试,然后分别做了 1200, 1600, 2000 个并发用户,执行 15 分钟的压力测试。得出系统最佳的并发用户数,了解系统的性能瓶颈。

5、监控 linux 服务器的 CPU,内存,IO 等资源。(非必须步骤)

6、分析性能测试结果(聚合报告)。(如果有问题,收集相关的日志提单给开发修改)

7、开发优化代码好后,回归性能测试。(非必须步骤)

8、编写性能测试报告。

PS: 1.性能测试一般指接口性能测试,负责接口测试的同事可能顺带做性能测试,小自研公司往往是这么干的。

2.大公司有专门的性能测试组,所有项目组只要有性能测试需求就丢给性能测试专家组来完成。

3.简单的性能测试,只要会使用 jmeter 就可以做。性能调优一般由开发或性能测试专家组来做。

4.真正专业的性能测试包括:性能测试、性能监控、性能调优。(需要对底层比较了解,对人的综合素质要求高)

5.性能测试的结果图表一般是给产品经理和开发分析的。有些项目组,性能测试其实是由某个开发兼着做。

*性能测试会吗?

会,很简单,主要使用 jmeter 进行接口的压力测试,然后查看聚合报告就行。

PS:聚合报告等得到的性能指标一般不需要我们分析,给产品经理和开发分析,我们只是通过测试而获得结果。

性能测试在哪里做?

我们会搭建一套独立的性能测试环境进行测试(一般由开发来搭建)。

然后在本地使用 jmeter 发压。(也可以选一台 linux 服务器安装 jmeter 来发压)

PS:测试环境默认指服务器的项目环境。安装 jmeter 之前需要安装 jdk 并配置环境变量。

*性能测试什么时候做?

功能测试之后,系统比较稳定的时候再做(回归测试时候做,产品经理会指定几个重要的接口)。

(性能测试一般有专门的人做,我也会,我在小自研公司的时候,自己做过登录接口、下单业务接口的性能测试)

PS: 1.性能测试不是为了发现功能 bug,是为了验证自己系统是否满足性能需求,发现性能瓶颈,提高系统的性能。

2.小项目,不太稳定或者用户量比较少的项目(注册用户数达不到 1w 的项目),我们一般不做性能测试。

*性能测试的目的（意义）或为什么要做性能测试？

评估当前系统能力是否满足性能需求，寻找系统的性能瓶颈从而优化性能。

PS：开发优化代码，运维优化 tomcat、mysql 的配置（一般是运维搭建集群环境和装各种软件），硬件运维提高硬件的配置。

性能测试的用例怎么设计？

- 1.先性能需求分析，挑选了用户使用最频繁的功能来做性能测试，编写性能测试用例；
- 2.性能用例有名称、测试步骤、场景设计、执行时间、预期结果；
- 3.执行时候，再把性能测试用例转为 jmeter 性能测试脚本，添加线程组，填写接口 url、请求参数，并优化，包括：参数化，断言，接口关联(边界提取器)等。

*性能测试，发现过什么问题？(如何识别性能瓶颈)

- 1.发现 CPU 使用率超过 80%。

优化措施：因为是查询数据的场景，表数据量比较大，所以建议开发做了索引，减少数据库服务器的资源消耗。

- 2.发现内存使用率过高，可能内存溢出发生宕机。

优化措施：建议开发加大服务器的缓存；及时释放内存中的无效对象。

- 3.发现 TPS(每秒事务数)达不到要求，且事务失败率过高。

优化措施：加大数据库连接池的最大连接数，加大服务器缓存。

- 4.响应时间比较长。

优化措施：优化前端图片大小格式，优化查询的 SQL 语句，加大前端或服务器缓存。

- 5、性能问题：当并发线程数增加到 100 时，服务器报错 500，并抛出“连接异常”：太多的连接。

原因：mysql 数据库默认配置的连接数是 100，当 java 连接数据库的线程数接近 100 时，报错。

如何解决：把 mysql 数据库配置文件 my.ini 中的最大连接数 max_connections 改大一点。

(10000，宜大不宜小)

HTTP Status 500 – Internal Server Error

Type 异常报告

消息 Data source rejected establishment of connection, message from server: Too many connections

描述 服务器遇到一个意外的情况，阻止它完成请求。

数据源(数据库)报错：太多的连接

Exception

```
com.mysql.jdbc.exceptions.jdbc4.MySQLNonTransientConnectionException: Data source rejected establishment of connection
    sun.reflect.GeneratedConstructorAccessor48.newInstance(Unknown Source)
    sun.reflect.DelegatingConstructorAccessorImpl.newInstance(DelegatingConstructorAccessorImpl.java:45)
    java.lang.reflect.Constructor.newInstance(Constructor.java:422)
    com.mysql.jdbc.Util.handleNewInstance(Util.java:406)
    com.mysql.jdbc.Util.getInstance(Util.java:381)
    com.mysql.jdbc.SQLError.createSQLException(SQLError.java:984)
```

- 6、性能问题：当并发线程数逐步增加到 200 时，服务器出现了 500 报错，并抛出“sql 异常”：你不能操作一个已关闭的 sql 语句对象。

原因：语句对象被关闭后又重新使用了。

(或数据库连接对象关闭后，还没被“jvm” java 虚拟机回收，就被重新获取使用了)。

如何解决：是开发解决的，开发优化了一下获取语句对象的方法：获取语句对象后，先判断该对象是否已关闭或为空 null，确保每次获取的连接对象都是正常的。

(或从连接池获取连接对象后，先判断该对象是否已关闭或为空 null)

HTTP Status 500 – Internal Server Error

Type 异常报告

消息 java.sql.SQLException: You can't operate on a closed Statement!!!

描述 服务器遇到一个意外的情况，阻止它完成请求。

Exception

你不能操作一个已关闭的sql语句对象

```
java.lang.RuntimeException: java.sql.SQLException: You can't operate on a closed Statement!!!
    com.it.util.BaseUtil.checkToken(BaseUtil.java:89)
    com.it.json.servlet.AccountInfoServlet.doPost(AccountInfoServlet.java:66)
    javax.servlet.http.HttpServlet.service(HttpServlet.java:660)
    javax.servlet.http.HttpServlet.service(HttpServlet.java:741)
    org.apache.tomcat.websocket.server.WsFilter.doFilter(WsFilter.java:53)
```

7、当设置的并发用户数远远超过服务器能承受的并发用户数时，服务器一般会报错 503（服务器超载）。

性能测试遇到过什么 bug

1. 借贷项目：

1.1 放款支付接口的并发问题

现象：在并发请求下，相同借款订单号可以成功放款多次，导致生成多条相同订单号的还款计划。

解决方案：利用锁机制或消息队列，放款前，校验一下当前的放款处理状态(包括中间状态)。如果发现是处理中或者处理完毕，则拒绝此次请求。

1.2 还款接口的并发问题

现象：相同还款计划 id 可以成功还款多次，导致生成多条相同 id 的已还款记录。

解决方案：利用锁机制或消息队列，还款前，校验一下当前的还款处理状态(包括中间状态)。如果发现是处理中或者处理完毕，则拒绝此次请求。

2. 电商项目：

2.1 优惠券领取的并发问题

现象：我的 xx 项目，多人同时领取时，会领取到相同 id 的优惠券，然后其中某一使用人使用该优惠券，所有相同 id 的优惠券都会被核销，造成其他人无法使用。

原因：生成和发放优惠券 id 的代码逻辑有问题，多人同时领取时，对每个人应该发放不同 id 的优惠券。已经领取过的优惠券 id，给个已被领取的状态码。

解决方案：开发优化调整代码。

2.2 秒杀活动的并发问题

现象：秒杀商品时多用户同时下单，出现超卖现象（购买数量超过库存数量），比如库存为 10，100 个用户同时下单一个商品，成功下单的超过 10 笔。

原因：扣库存的逻辑存在问题，秒杀属于立即购买，无需加入购物车再购买，订单结算并下单时应该对库存进行锁定。

解决方案：开发优化调整代码，在秒杀下单时扣库存，取消订单时则恢复库存。

2.3 多用户同时下单, 偶尔会存在有些人的订单无法生成

现象: 多用户同时下单, 偶尔会存在有些人的订单无法生成, 查看日志发现主键重复异常。

原因: 下单时, 绝对并发用户数超过 50 时, 会生成了重复的订单号。

解决方案: 开发调整逻辑, 采用 redis 生成订单, 在 redis 中避免重复的订单号。

2.4 在并发测试时, 用户查询造成了内存溢出

现象: 做并发测试, 查询数据时, 服务器的内存消耗非常高, 在服务器日志中看到了内存溢出的异常。

原因: 查询时获取了数据所有字段包括商品详情, 而且没有对数据进行分页查询, 造成内存消耗过多;

解决方案: SQL 中去除不必要的字段, 并且对数据进行分页查询;

*性能测试指标有哪些?

1. 事务平均响应时间 (<3 秒)、事务成功率 (>98%)、吞吐量 throughput ; (聚合报告查看)

2. CPU、内存和 IO 的资源使用率(<75%); (监控服务器代理来查看)

PS : 单用户(单线程)访问的响应时间为 300ms 左右, 多并发用户(多线程)访问的响应时间为 3s 左右。

*如何分析性能测试结果?

1. 主要查看聚合报告和服务器资源使用图, 查看响应时间、事务成功率、CPU、内存、IO 使用率是否满足需求;
2. 如果出错率达到了总请求数的 3%, 我们会检查是什么原因导致的, 修改好后, 重新测试;
3. 如果出现了性能瓶颈, 比如响应时间, 或者 CPU 使用率不达标, 我们会从服务器上导出日志, 分析是哪个地方导致响应时间过长, 如果分析不出来, 就叫上开发一起讨论, 确定问题后, 再提单给开发修复, 修复好了就进行回归测试。

*分析一下影响性能的因素 (性能瓶颈如何分析? 或 响应时间不达标, 一般是什么问题?)

1. 首先考虑网络问题、jmeter 客户端问题:

网络卡可优化网络, 配置低可调大 jmeter 对 java 虚拟机 (jvm) 的内存, 硬件不行可换电脑测或使用服务器来测。

2. web 服务器问题: tomcat 默认配置的线程数 200 可改大, 线程模型默认的 nio 同步非阻塞可改为 aio 异步非阻塞。

3. 代码问题: 接口代码中流对象没及时关闭, 对数据库的连接数过多且没及时关闭, 或操作了已关闭的语句对象。

4. 数据库问题: 库表冗余数据多, 产生了慢查询, 通过慢查询日志, 找到相关表, 清理数据或建立索引来优化。

还可在 my.ini 配置文件修改最大连接数。

5. 中间件问题: redis 缓存的数据太多, 可调大 redis 占用的内存。还可调大 tomcat 服务器的 java 虚拟机内存。

有验证码的功能, 怎么做性能测试?

1) 暂时关闭验证码功能。

2) 设置万能验证码。

*你们项目的最佳并发用户数是多少?

1. 大概 2000。我们当时从 1000 做到 2000 个并发用户的时候, 查询功能的响应时间超过了性能指标 2 秒多, 原因是有几个表的索引建得不合理导致的, 重新建索引后, 问题就解决了。

2. 我们当时做到 2000 并发用户后, 系统资源使用率 (cpu、内存、io 等) 也达到了 75%。

就没再继续增加用户量了。因此, 我们系统的最佳并发用户数就是 2000。

PS : 1. 物业、OA、ERP、CRM 等内部员工使用的管理系统, 设计的并发用户数一般 200 就足够了。

2. 一般的互联网项目，设计的并发用户数一般 2000 就足够了。

吞吐量和并发用户数的关系的怎样的？

一个项目的吞吐量是有个极限的，就像一条马路，假设最多能同时通行 5 辆车，那么这条马路的最大吞吐量就是 5 辆车；当并发用户数没达到吞吐量的极限时，随着并发用户数增加，吞吐量就会增加，当并发用户数达到吞吐量的极限后，吞吐量会保持平稳。

*如何监控服务器的资源（就是怎么安装）？

1. 首先在 jmeter 上安装监控服务器资源的插件 PerfMon 并在线程组添加该监听器。
2. 然后在 Linux 服务器上安装 ServerAgent 代理工具，对该工具进行解压、**赋权**，最后开启代理服务。
3. 在客户端运行 Jmeter 脚本，就可以通过 PerfMon 插件查看服务器的资源了（CPU/内存/IO 等）。

PS：1. 公司服务器一般安装 linux 操作系统，而不是 windows 系统。

2. 很多中小型公司的生产环境会选择**租用云服务器**，比如**阿里云、腾讯云、华为云**。（自建机房的成本高）

*如何监控服务器的性能指标（每秒事务数、活动线程、响应时间等）？

在 jmeter 中安装插件 Standard Set 并添加对应指标的“监听器”即可监控这些指标。（一般用自带的聚合报告就够了）

*如何查看聚合报告

聚合报告不用安装插件，jmeter 自带此功能，聚合报告主要查看响应时间、事务成功率、吞吐量 throughput。

*性能测试的分布式环境(多负载机压测)怎样搭建？

性能测试分布式环境搭建还是有点复杂的，我自己整理了一个文档。和你大致说一下搭建的思路吧：

- 1、在控制机的 Jmeter 属性配置文件中先配置从机的 ip 和端口号（端口号默认为 1099）。
- 2、在每台从机启动 jmeter 的服务 jmeter-server.bat。
- 3、在控制机的 Jmeter 上打开.jmx 脚本，设置并发线程数，并设置“远程启动所有”。这样就能控制多台从机和控制机一起测试。（每台从机和控制机是一样的并发线程数）

PS：可能有些人称为“jmeter 集群环境”，指的就是“jmeter 分布式环境”。

为什么要做分布式压力测试？

一台 jmeter 客户端电脑能创建的并发用户数和发出的请求数是有限的。（一般最多 2000-5000 个线程数(用户数)）

PS：1. 有时候性能测试的瓶颈在客户端，因为单个客户端能创建的并发用户数是有限的。

2. 公司路由器也是性能测试的客户端瓶颈。因为一台路由器能处理的请求数和网络速度也是有限的。

一台电脑如何同时发出 10w 个请求

首先一台 jmeter 客户机能发送的请求是有瓶颈的(配置好点的普通电脑能发出 5k 个请求，内存 16GB+CPU2.5GHZ)。所以此时需要搭建 jmeter 分布式环境，让一台 jmeter（控制机）同时控制多台 jmeter（从机）来一起发压。

PS：当并发线程太多了(超过 10w)，路由器也是个性能瓶颈，可能需要跨局域网(跨路由器)搭建 jmeter 分布式环境。

你们性能测试做的是前台还是后台？

PS：现在很多系统是前后端分离的，前后端都是独立的 web 系统。

以前传统的 web 项目，前后端合在一起作为一个项目。

回答：我们做的是后端服务器(接口)的性能测试。(前端页面文件一般没有性能测试的需求, 有些大公司)

*如何实现 200 用户的真正并发？

先设置并发用户数为 200, 再针对请求添加同步定时器(集合点), 默认情况下, 当集合到了 200 线程则会并发执行。给以服务器瞬间的压力。

*并发用户数怎么确定的？

1. 并发用户数, 产品经理主要根据 2-8 原则估算, 百分之 80 的操作次数在百分之 20 的时间内进行。

比如一天的高峰期 8 小时查询订单次数在 100000, 问并发用户数？

总用户 TPS: $(100000 * 80\%) / (8 * 60 * 60 * 20\%) = 13.89 \text{ 次/s}$

单用户 TPS (估算): $1 \text{ 次}/30\text{s} = 0.033 \text{ 次/s}$

并发用户数: $13.89 / 0.033 = 421 \text{ 个}$

2. 测试时候, 先从单用户单功能的循环的基准测试开始, 逐步加压, 分别做了 1200, 1600, 2000 个并发用户, 执行 15 分钟的压力测试。系统资源使用率 (cpu、内存、io 等) 也达到了 75%。从而得出系统最佳的并发用户数, 了解系统的性能瓶颈。

PS: 性能指标一般由产品经理根据系统的业务量进行估算的, 且性能需求书上会描述出来。

你们的性能测试需求哪里来的？

我们的需求文档上就有的, 不过有时候不太合理, 我们可能需要和 BA 产品经理进行讨论。

比如, 我之前做了一个物业公司内部用的物业管理系统的性能测试时, 要求并发用户 500 人, 物业公司整个公司办公人员都没有 500 人, 很明显, 做 500 人并发是没有必要的, 后来, 我们只做了 200 人并发。

*压测过什么接口 (三选一)

电商: 登录接口、添加购物车接口、下单接口、订单查询接口、购物车查询接口。

借贷: 登录接口、申请借款接口、放款接口、还款接口、借款订单查询接口。

物业: 登录接口、查询小区信息接口、查询工单接口、添加工单接口。

*如何压测添加购物车接口 (或如何压测其他 xx 接口)

错误回答: 添加一个线程组, 设置并发数, 先添加一个登录的 http 请求, 从里面提取 token 值, 再添加一个添加购物车的 http 请求。(因为登录接口被迫进行了压测)

正确回答: 1. 先添加一个线程组, 并发数默认 1, 里面通过登录请求提取 token 值, 且设为全局变量。

2. 再添加下一个线程组, 里面新建一个添加购物车的 http 请求, 获取全局变量 token 的值, 并作为请求头参数进行登录身份的验证。仅在该线程组设置并发数即可。(仅添加购物车接口进行了压测)
(有必要的話, 还可以优化脚本: 设置请求默认值、用户自定义的变量、csv 参数化)

性能测试会做混合场景的吗, 还只是单场景

单场景做的多点: 测试一个线程组的单登录 或 单添加购物车 或 单下单。

PS: 混合场景: 设置多个线程组, 一个线程组测登录, 下一个线程组测添加购物车, 下一个线程组测下单, 同时进行。

*如果没有接口文档, 怎么做接口测试或者性能测试

1.很简单, 可以使用 fiddler 或 F12 抓包, 抓取浏览器页面点点点的请求, 从而了解接口的先后调用逻辑及其依赖和接口需要的请求参数。

2.也可以使用 **jmeter 代理服务器**来录制浏览器的请求脚本:

先添加一个线程组, 在下面添加 Cookie 管理器。再添加一个 HTTP 代理服务器, 其中的目标控制器选择该线程组。浏览器设置代理服务器 (填写 HTTP 代理服务器的 ip 和端口, 习惯默认 8888) 后, 打开浏览器点点点, 发出的请求都会被 jmeter 录制下来。(还可以使用过滤功能实现精准捕捉)

PS: 1、前后端分离项目的前端如何调用后端接口? (站在测试角度, 和前后端合一的传统项目没有什么区别)

<https://blog.csdn.net/ChaoticNg/article/details/114586609>

2、如果是微服务项目后端之间的调用, 或不同系统之间接口的调用, 则必须要提供接口文档, 因为没办法抓包。

***如果让你一个人负责性能测试, 可以吗**

1. 没问题的, 性能测试就是用 jmeter 设置并发线程数(一个线程就是一个虚拟用户), 针对接口进行并发测试。

查看聚合报告, 查看服务器的资源使用率, 最后分析性能指标是否满足需求并且发现系统的性能瓶颈。

2. 我对物业项目的登录接口、搜索工单接口、添加工单接口都做过性能测试 (xx 商城的登录接口、搜索商品接口、下单接口)。这些操作我在公司都有整理过操作文档。

一台电脑上如何安装多个 tomcat 服务器

很简单, 把另外一台 tomcat 服务器的默认端口号 8080 修改为其他值就行了。

(因为同一台主机上, 运行程序的端口号不能冲突)

PS: 具体在哪改: server.xml 文件。最好把关闭程序默认的端口号 8005 也改一下, 防止关闭一台 tomcat 会自动把其他的 tomcat 也关闭了。

性能测试方法(性能测试分类)

基准测试、并发测试、负载测试、压力测试、稳定性测试、配置测试。(也称为性能测试策略)

*性能常识

1. 借贷系统或者普通 B2C 商城并发数达到 2000 算比较高的 (虽然淘宝支持 10 万+)。一般 tomcat 服务器默认支持 200 个并发, 可以修改配置文件 tomcat/conf/server.xml、tomcat/bin/catalina.bat 增大并发数。

2. 默认情况下, Windows 中每个进程的线程数默认不允许超过 2000, Linux 中每个进程的线程数默认不允许超过 1000 (Tomcat 服务器就是一个进程)。

3. 当并发数超过 250, 单台服务器主机搞不定了, 一般需要考虑集群, 使用 Nginx+tomcat 的集群设计。(mysql 默认最大连接数为 100, 也可以修改配置文件 my.ini 增大这个值)

4. Nginx 用于负载均衡和请求分发, tomcat 用作 web 服务器。我们系统架构是 Nginx+tomcat+redis。

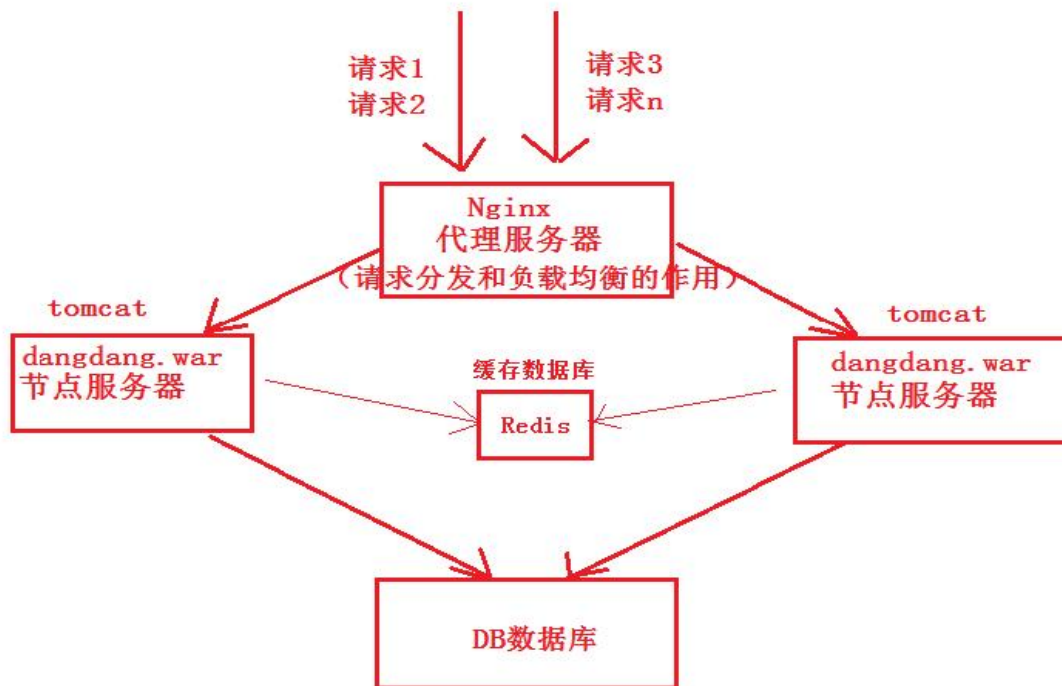
5. 面试官问起我们借贷项目或者电商项目的并发用户数, 就说并发用户数设计为 2000, 共有 8 台节点服务器, 吞吐量 throughput: 500 个请求/秒左右, 每秒事务数 tps: 大概 500。

系统使用了 redis 缓存技术 (一台 redis 服务器), 提升了查询库表的效率 (第一次查询订单表数据, 去数据库查并保存一份在 redis 中, 后面每次查询请求都先去 redis 中查, redis 有的话, 就不用去数据库查询了)。

6. 物业项目等 ERP、CRM、OA 等管理类项目, 一般是企业内部使用的系统, 不是对外的互联网项目。

并发用户数设计为 200 足够了, 一台服务器主机就能搞定。

web服务器 (tomcat) 集群设计 (cluster)



*你们商城的日均 ip 访问量、业务量 (交易量)、并发用户

- 1.注册用户 100 万, 活跃用户 10 万。日均 ip 访问量 1 万左右。(京东是 581 万, 苏宁易购是 51 万, 2016 年数据)
- 2.日均业务量 (GMV 交易量), 说不太了解就行 (大概 100 万人民币)。
- 3.并发用户数 (并发量) 一般 2000 左右。

PS: 1. 其他对外的互联网项目可以借鉴着说, 指标建议说成商城的 1/2 或 1/10, 并发用户数说 1000 或 200。

2. 借贷项目的注册用户 50 万, 活跃用户 5 万。日均 ip 访问量 0.5 万左右。(这些是运营了解的数据, 我是小测试)

3. 借贷或者商城 app 的下载量: 回答 100 万左右。

*你们系统架构是什么、有几台服务器

- 1.电商项目、借贷项目: 我们系统架构是 Nginx+tomcat+redis, 有 8 台服务器, 设计的并发量为 2000。
- 2.物业项目等 ERP、CRM、OA 等管理类项目: 我们系统有 1 台 tomcat 服务器, 设计的并发量为 200。

PS: 如果不是强调性能方面的架构, 从整个系统, 我们可以简单回答: Linux+Java+Tomcat+Mysql。

*你们用到的中间件

中间件: nginx (用于请求分发和负载均衡)、tomcat (提供 http 的 web 网络服务)、redis (用于缓存数据)。

nginx、tomcat、redis, 这种处于浏览器和数据库之间的软件称为中间件。

*你们的技术栈是什么?

- 1.前端: vue、react、angular、jQuery; (基于 js 语言) (说出一个即可)
- 2.后端: springBoot/springCloud、spring、springMVC、mybatis/MyBatisPlus、redis/Kafka/MQ、docker;
- 3.这些是开发需要用到的框架 (相当于 python 编程需要用到的模块), 我作为测试, 了解的有限。

PS : springBoot 框架写的项目为单体程序，不需要另外放到 tomcat 服务器的 webapps 里面。

因为这些框架自带 tomcat 服务器模块。springCloud 是基于 springBoot 的微服务技术。

5.Fiddler 抓包

*Fiddler 如何使用

1. 很简单，打开 fiddler 后，浏览器会自动设置 fiddler 为代理服务器（自动配置 fiddler 的 ip 和端口号）。
fiddler 会自动抓取浏览器的 http 请求会话；
 2. 点击对应的请求会话，就可以查看 http 协议内容（请求和响应），我一般看 raw 原始内容（未经过格式化的）。
- PS：抓包工具 **charles**：默认端口也是 8888，也能进行接口测试，也比较流行，苹果电脑 mac 喜欢安装 charles。

*为什么使用 fiddler

使用 fiddler 可以抓取 http 协议的请求和响应，便于分析 bug 是前端问题还是后端接口问题。还可以打断点修改请求或响应数据。

*什么时候需要用到 Fiddler 抓包工具，Fiddler 可以用来干嘛？

- 1) 当测试发现缺陷时，使用 fiddler 抓包，查看请求和响应，定位该问题是前端还是后端的问题；
- 2) 抓取 app 的请求响应、抓取 python+requests 或 jmeter 的请求响应；
- 3) 模拟弱网环境；
- 4) 做安全测试，检测敏感信息是否加密，拦截篡改数据。

PS：工作中一般使用 **F12**，查看 http 请求响应比较方便点。开发更喜欢使用 F12，fiddler 可能都不用。

用浏览器怎么抓包

电脑浏览器自带 F12 抓包工具，软件开发工程师常用 F12 功能于查看电脑 web 页面的请求和响应来定位问题。

Fiddler 怎样抓 Https 的包

- 1、先安装 fiddler 安全证书（让浏览器信任 fiddler）；
- 2、点击 fiddler 的 https 设置；
- 3、勾选上所有选项，更换证书。重启 fiddler。

*Fiddler 如何对手机 app 抓包？

- 1、很简单，手机与 fiddler 所连的电脑连接到同一网络；
 - 2、在 fiddler 设置监听端口号 8888，并允许远程终端连接；
 - 3、在手机上填写代理服务器的地址和端口号。这样的话，手机访问互联网的请求都会经过 fiddler 而被抓取。
- PS：需要提前通过手机浏览器访问 fiddler 所在的 ip:8888 而下载安装 fiddler 安全证书，用于手机信任 fiddler。

*Fiddler 如何打断点(拦截数据、串改数据、**改包**)？

点击 Automatic Breakpoints 自动断点功能，选择 Before Requests 请求前打断点或者 After Response 响应后打断点。（不用的时候，选择 Disabled 禁用）

*fiddler 如何抓到自己想要的包（请求过滤）？

答：利用 fiddler 的 filter 过滤器功能，勾选“ 仅仅显示 URL 包含” 。

Fiddler 的抓包原理

fiddler 在客户端和服务端之间扮演代理的角色, 通过监听端口号(默认是 8888), 监听本地发出的请求和服务端响应的结果。

*如何模拟弱网环境进行测试?

1. 可以使用 fiddler (Simulate Modem Speeds) 的限速功能。

打开 fiddler 的脚本编辑器, 找到 delay 延迟的地方, 修改数据即可。

2. 我们弱网的标准是, 请求网速大概 3kb/s, 响应网速大概 6kb/s, 和默认设置差不多。

fiddler 或者 F12 开发者模式查看的信息主要有什么

参考答案: 请求信息和响应信息, 请求有请求头和请求体, 响应有相应头和响应体。

为什么用 fiddler 抓取 app 的包

答: app 对接口发出的请求和得到的响应数据需要通过 fiddler 抓包来查看。便于分析 bug 是前端 app 问题还是后端接口问题。

PS: 抓包工具还有 charles, 苹果电脑习惯用 charles。和 fiddler 没有什么区别。

6.APP 测试

*app 测试和 web 端测试的区别是什么？

- 1 系统架构：web端系统，更新服务端后，不需要更新客户端；APP如果更新了服务端，客户端也要更新并测试
- 2 兼容性：Web端要考虑不同的浏览器内核进行测试（chrome、Firefox、IE）。
APP的兼容性要考虑选择主流的机型，不同的分辨率、尺寸，以及不同的操作系统；
- 3 App要考虑交叉中断事件测试，还有安装，卸载，前后台切换测试；
- 4 App还要考虑界面操作，如：横竖屏切换，多点触控，事件触发区域。

app 测试有哪几种环境？如何搭建？

- 1.本地环境：app 安装的手机环境和电脑搭建的 APP 自动化测试环境（比如安卓 SDK 等等）。
- 2.服务器环境：war 包部署的服务器，可通过浏览器访问，也可以通过 app 去访问。（访问的是服务器后端接口）

*app 的兼容性怎么测试，测试了哪些机型和哪些版本？

- 1) 不同的手机，如：华为，小米，vivo，OPPO
 - 2) 不同操作系统的兼容性是否适配，如：最近几个版本的安卓系统，最近几个版本的苹果 ios 操作系统
- PS：实际工作中，先使用一种真机来测试，后面再测主流机型的兼容性。（模拟器是用来学习的）

app 项目做了多久？

一直在做，现在 app 的主体需求已经完成了，后期都是一些零零散散的需求，测试工作量比较少。

你负责 APP 的哪些模块

基本所有的模块都是我测。（因为 app 功能比较简单单一，所以一个测试工程师就可以负责所有模块）

PS：现在 app 主要有购物，美食，交友，支付，交通，视频，浏览器，打车类，炒股类，地图类，网络电话，根据不同类别讲述 app 的主要功能，同一类的 app 功能基本一致。

*ios 的 app 怎么测试

苹果 iso 系统比较封闭，工作中测试主要是手动点点点，测功能为主。

PS：adb 和 monkey 是安卓的测试工具。Monkey 是安卓系统自带的（非图像化工具），adb 是本地工具。

*H5 测过没

答：有测试过的，H5 就是手机网页测试，主要是功能测试，点点点。

- 1、H5 网页跨平台，兼容性强；（H5 还可以指高仿的 app，不需要通过应用市场，直接在手机本地安装）
- 2、运行速度慢，耗网速，用户体验受限；
- 3、调用移动底层硬件设备效果不是很好，无法在本地保存用户数据；

PS：1、H5 是 html 第五代语言，2014 年正式推出。小程序是基于 H5。

2、移动端包括：app、小程序、H5（手机浏览器访问的网页）。这些都是页面功能测试为主，有测试过的。

安卓 app 还可以用 monkey 命令测试一下稳定性。

*小程序测过没

答：有测试过的，就是微信里面的网页测试，主要是权限测试 和 功能测试，也是页面点点点。

- 1、程序分为三种版本类型：开发版，体验版，正式版。开发版和体验版无需审核，需要给微信号配置权限，通过扫小程序的二维码才能访问。大小限制 2M。
- 2、目前小程序不支持直接分享朋友圈，只能分享微信好友。所以很多小程序都采取了“曲线救国”的方式，通过生成带有小程序码的图片，用户可以退出小程序将图片发布到朋友圈。
- 3、页面路径最多只能十层；基于 H5 且依赖于微信，即点即用（用完即走），用户使用成本低；打开速度比 H5 快，慢于原生 app。（微信小程序是 2016 年 12 月份推出的，2017 年开始发展）

*adb 命令有哪些？

- 1) 连接 Android 设备：adb connect 设备名
- 2) 查看连接设备名：adb devices
- 3) 安装软件：adb install 路径/名称
- 4) 卸载软件：adb uninstall 软件名
- 5) 查看日志：adb logcat -v time
- 6) 从本地上传到安卓手机：adb push \本地路径\ /APP 路径/
- 7) 从安卓手机下载到本地：adb pull /APP 路径/ \本地路径\

monkey 如何做回归测试（稳定性测试）

发现 bug 后，一般需要按照之前的轨迹去跑 monkey，命令是：monkey -p 包名 -s seed 值 -v 次数。

*app 稳定性(性能)测试怎么做？

手机端的性能：我们是用 monkey 做手机端 App 的稳定性测试的，使用 monkey 跑 10 万次，看它会不会出问题，如果出了问题，我们再定位原因，具体的做法是这样的：

- 1.在跑 monkey 前，先清空手机的 logcat 日志：adb logcat -c
- 2.接下来，获取 logcat 日志：adb logcat -v time > d:\share\logcat.log
- 3.打开新的 cmd 窗口，使用 monkey 运行被测应用：

```
adb shell monkey -v -p your.package.name 100000 > d:\share\monkey.log
```

- 4.测试完成后查看 monkey 日志，如果说它跑的次数 Events injected 跟我设的次数 count 不一样，就说明 monkey 中途跑失败了。

那我就要去看看 logcat 日志有没有 **NullPointerException**(空指针异常)，或 **anr**(应用无响应)的关键字。

如果有 **NullPointerException**，就表示 app 在测试过程中 crash(崩溃)了，然后把 **NullPointerException** 附近的日志截取下来，发给开发定位；如果有 **anr**，表示 app 在测试过程中出现了“应用无响应”，我们要把/data/anr/traces.txt 文件取下来，再把 anr 的进程号对应的日志信息发给开发定位问题。

Android ANR 日志路径在哪

/data/anr/traces.txt

*为什么要用 monkey 进行稳定性(性能)测试或 monkey 测试的目的和意义？

通过 monkey 跑自动化，可以发现持续点击运行下的 app 是否存在 null point(空指针异常)或 anr(应用无响应)的性能问题。

app 测试环境搭建(android)

- 1.先搭建服务器的项目环境。（用于提供接口服务。流程同 linux 下搭建当当网测试环境）
- 2.再搭建本地环境：安装 jdk 和 Android sdk，配置 java 环境变量。最后把 app 安装到真实手机上进行测试。

当 monkey 异常终止，我如何协助开发定位问题？

1. 运行 monkey 做稳定性测试的同时，需要把应用的 logcat 日志取下来，monkey 在运行中出现异常，一般是两种原因导致的，一个是 crash 程序崩溃，一个是 ANR 程序没有响应；
2. 当 monkey 异常终止，就进入 logcat 日志，搜索关键字“anr in”，如果存在该关键字，说明 app 出现了 ANR 异常，接下来把/data/anr/目录下的 trace.txt 文件取下来，将出现问题的进程号对应的日志发给开发定位；
3. 如果存在“null pointer”空指针异常该关键字，说明 app 出现了 CRASH 异常，就把 null pointer 异常关键字上下范围的日志截图发给开发定位。

monkey 会出现哪些异常，有哪些原因

APP 出现 ANR 的原因：（应用无响应）

- 1、线程阻塞
- 2、内存不足
- 3、CPU 满负荷(由于现在的手机基本都是 8 核 CPU，所以基本不会出现 CPU 满负荷的情况)

APP 出现 CRASH 的原因：（闪退/崩溃）

- 1、空值指针异常
- 2、数组越界异常
- 3、内存不足错误
- 4、CPU 满负荷(由于现在的手机基本都是 8 核 CPU，所以基本不会出现 CPU 满负荷的情况)

*如何判断 app 的 bug 是客户端问题还是后台问题

一、客户端问题（前端问题）

- 1、文字，图片有误，无法输入，按钮不可用；
- 2、抓包信息显示客户端发送的信息有误，或状态码 401、404。

二、服务端问题（后端问题）

- 1、通过抓包检查服务器接口返回的信息，如果有误，就可以断定是服务器的问题；

2、客户端向服务器接口发送信息后，服务器响应 500、503 等。

PS：这个要根据业务来，一般数据本身有误则属于后台问题。我一般做法是有问题提给 app 开发，他们知道是 app 的问题还是后端接口的问题。

*app 项目做了多久

一直在做，现在 app 的主体需求已经完成了，后期都是一些零零散散的需求，测试工作量比较少。

app 和 web 端同步更新。

*你这个 app 测试人员有几个？怎么分工？

2 个，按测试的手机类型分工，每个人负责几种测试机型(有些人负责 ios 测试)，每个人都要测试 app 的所有测试点。

*app 接口测试做过嘛

很简单，做过，app 接口测试和 web 端接口测试没有区别，都是服务器的接口。

*app 的测试点（或者测试方法）



借贷（或 B2C 商城）的 app 多久更新一次？

答：app 和 web 端同步更新，小版本 1 个月一次。

（ios 的 app 还需要审核 7 天左右，审核期间，测试一般在进行下一版本的前期工作，不可能干等）

app 埋点介绍

在 app 上或者 web 网站上，记录用户浏览操作行为，统计用户行为数据，可以为个人用户更精准地推送个性服务。

PS：埋点分析，是网站分析的一种常用的数据采集方法。植入多段代码，追踪用户在平台每个界面上的系列行为，

事件之间相互独立（如：打开商品详情页——选择商品型号——加入购物车——下订单——购买完成）。

数据埋点怎么测试？

1. 数据埋点一般就是指在软件中的页面里埋藏了一些接口, 去记录一下用户的访问或操作信息的数据, 然后上传到数据库中。
2. 在测试数据埋点时, 主要就看每个埋点是否生效, 以及记录的数据是否准确, 是否会出现漏统计或者重复统计的现象。

app 如何保持长登陆状态

1. 第一次输入用户名密码登录成功时, 服务端生成一个唯一 token (令牌) 给 app, app 每次发出请求都携带这个 token 去服务端, 服务器解密 token, 获取其中的用户名和过期时间, 如果验证成功, 则允许当前请求。如果验证失败, 则重新登录。
2. 当服务端收到 token 的过期时间靠近当前时间, 则给个新的 token。(服务器本身不会保存此 token, 此 token 一般是 jwt Token。

*app 常识

1. app 的 ui 自动化使用 python+appium 技术, appium 是高仿的 selenium, 两者差不多。前者仅限于 app 自动化。
2. 安卓 7.0 以上后默认不信任用户自行安装的证书文件、如果需要 fiddle 抓包, 就要把证书放到系统目录下。
或者安装 VirtualXposed + JustTrustMe 两个 apk 包, 这个 apk 其实可以理解为手机内装一个虚拟机, 然后在虚拟机内运行我需要抓包的应用, 该虚拟机内是不需要证书验证的。然后就可以随心所欲的抓包了。
3. 安卓操作系统常用的版本是 5.6。
4. 工作中基本不用虚拟机, 用的是真机去测试, 虚拟机主要用于学习。
5. Android 四大组件 (java 框架、模块): Activity 活动、Service 服务、Broadcast Receiver 广播接收、Content Provider 内容提供。这些是安卓 app 开发工程师用于编程的框架 (模块、库)。
6. app 测试是小页面测试, pc 端测试是大页面测试。都是功能测试点点点为主, 接口测试也都是服务端的接口。

7.其他

*pytest 单元测试框架了解嘛

- 1.很简单, pytest 和 unittest 差不多, 文件名以 test_ 开头或 _test 结尾, 类名以 Test 开头, 测试方法以 test_ 开头。
- 2.断言一般用 assert in, 参数化用 @pytest.mark.parametrize(), 还可以结合 allure 工具, 生成 html 格式的测试报告。我平时工作中用到的 pytest 的内容大概就是这些。

PS: 1.当 test 测试用例方法有 n 个, setup()和 teardown()就会执行 n 次;

2.不管测试用例方法有多少个, setup_class()和 teardown_class()只会执行一次。

3.setup_class()在 setup()之前先执行, teardown_class()在 teardown()之后执行。

4.test_xx 方法的执行顺序是根据方法的摆放顺序来。也可用装饰器 @pytest.mark.run(order=1) 定义执行顺序。

*unittest 和 pytest 框架的区别

- 1.unittest 是自带的框架, 自定义的测试类需要继承 unittest.TestCase, 属于侵入式设计。test 方法执行顺序按照方法名的字母顺序来。参数化一般使用单独下载的 ddt 模块, 断言一般使用 self.assertIn()。
- 2.pytest 是需要 pip install 下载的单元测试框架, 编写的代码比较简洁, 文件名要以 test_ 开头或 _test 结尾, 类名要以 Test 开头, 测试方法要以 test_ 开头。test 方法执行顺序按照方法的摆放顺序来。参数化一般使用 pytest 自带的装饰器。

@pytest.mark.parametrize(), 断言一般使用 assert in。

- 3.unittest 没有插件, pytest 有丰富的插件, 更易生成漂亮的测试报告。(unittest 不支持失败重试, pytest 可以)

PS: pytest 还可以运行 unittest。(pytest 比 unittest 更高级, 用的更多)。

python+requests+pytest+allure 接口自动化测试用了什么库?

- 1.主要使用了 requests、pytest, allure-pytest, yaml, os 模块。
- 2.pytest 用来编写测试用例和断言, requests 用来发送请求, allure-pytest 是用来生成测试报告的; yaml 用来存放测试数据, os 是用来执行 pytest 命令和 allure 命令。

*python 做接口测试时, 参数化怎么做?

如果是针对 pytest 模块, 在 test 方法上添加装饰器 @pytest.mark.parametrize(), 还可以引入外部 yaml 文件, 读取里面的二维列表数据。

PS: YAML 文件是什么: YAML 不是一种标记语言, 里面主要存放数据, 主要用于数据文件或配置文件。

支持的数据类型有对象(字典)、数组(列表)、纯量(基本数据)。

*pytest 如何生成 html 测试报告

很简单, 执行两条命令(可转为代码): **pytest** 测试文件路径 --alluredir 测试结果路径; # 执行测试文件, 得到结果
allure generate 测试结果路径 -o html 测试报告路径; # 把结果格式化成 html 报告

PS: 1. 提前安装好 allure-pytest 模块 和 allure 软件, 配置 allure 的 bin 目录的环境变量 (allure 是 java 程序)。

2. **pytest** 如何在测试报告中插入你想描述的数据: 在方法上定义装饰器,

@allure.title('自定义测试标题_登录测试')

@allure.story('描述_登录功能点')

@allure.severity('重要级别_高')

ddt 的原理是什么, allure 的原理是什么

1. ddt : data driver test 数据驱动测试, ddt 是 unittests 有关的参数化模块。

原理: 通过类上的@ddt.ddt 和 测试方法上的@ddt.data()/@ddt.file_data() 来引入数据

通过@ddt.unpack 进行数据解析并传给测试方法的参数名。

2. allure 是 java 写的一个工具, 用来生成测试报告的, pytest 通过 allure-pytest 模块来操作 allure 生成测试报告。

*svn 和 git 的区别, 你们公司用什么版本管理工具

svn 是集中式版本管理工具, 仓库的所有版本都存在同一台 svn 服务器上。

git 是分布式版本管理工具, 每个 git 客户端都有仓库的所有版本, 和 git 服务端没有区别。

svn 平常是用于项目组管理需求书、开发设计文档、测试用例文档、项目计划文档、测试计划文档等。

svn 在用户权限这块做的比较好, 每个员工都有自己的 svn 账号。开发协同写代码, 共享代码用的也是 svn。

在企业中 git 和 svn 的用途几乎一样, git 也有可视化的 Tortoise 乌龟界面, 方便操作。

git 使用起来稍微麻烦点, 开发进行代码管理用的多点。如果单纯管理工作文件, 推荐使用 svn。

我们公司文件管理使用 svn, 因为操作方便点。代码的管理之前用的也是 svn, 慢慢转 git 了。我们自动化脚本(py 或 jmx 脚本)也可以使用 git(或 svn)来管理, 公司内部搭建了一台 git 服务器。

(公司一般在 linux 系统上搭建 git 服务端, 大家用的其实是同一个用户名 git, 大家在本地自定义备注信息: 全局用户名和邮箱地址, 主要用于区分不同的用户。git 客户端使用 ssh 公钥方式访问 git 服务器)

*git 常见命令

git add #添加到暂存区

git commit -m # 提交到版本库。-m 给备注信息

git clone #克隆(拷贝)远程仓库到本地(第一次访问)

git push #推送到远程仓库

git pull #更新远程仓库到本地仓库(非第一次克隆)

*redis 缓存数据库(非关系型数据库/noSQL)

*redis 数据类型有哪些

字符串 string、哈希(字典 dict)hash、列表 list、集合 set、有序集合 zset(sorted set) 共 5 种。

*redis 是做什么的?

1. redis 是一个高性能的 key-value 键值对数据库。redis 属于非关系型数据库, 是 C 语言写的一个程序, 所有数据放到内存中, 起到缓存的作用。(redis 的出现, 很大程度补偿了 memcached 这类 key-value 存储的不足, 在部分场合可以对关系数据库起到很好的补充作用。Redis 的默认内存大小将取决于操作系统和其配置, 一般为 32MB, 64MB 或 128MB 等)

2. redis 可用于短信验证码、token、订单查询、搜索建议的缓存;

秒杀活动、各类榜单、购物车结算时库存扣减；在线用户的个人信息、用户的历史行为等等；

3. 有时缓存的脏数据会影响测试结果，需要 flushall 清除数据缓存。有时也要清除缓存，看看无缓存数据的情况下的系统使用情况。

在什么情况下会用 redis？

1. 有时需要连接 redis 缓存数据库查下短信验证码（get 13022221111#login）
2. 在测试的过程中，有些 bug 是后台缓存数据产生的问题，需要清空一下 redis 缓存（flushall）
3. 也有使用 redis 列表做消息队列的情况，那我们要重点关注产生的数据能不能完整的被消费掉。
4. 对热点数据进行缓存，比如搜索建议，秒杀，抢购，订单查询，排名等等。

redis 的穿透、击穿、雪崩

- 1、**redis 穿透**：当访问的数据在缓存中不存在，且查询数据库之后也不存在，就会导致系统每次都要访问到数据库，缓存没有起效果。解决方案：增加空缓存，访问数据库没有的数据也要在 redis 中也创建缓存。
- 2、**redis 击穿**：当大量请求的缓存数据同时失效，会有大量请求直接访问数据库而重新建缓存。
解决方案：将失效时间增加一定的随机值，让数据在不同时间失效。
- 3、**redis 雪崩**：缓存无法使用（比如缓存失效）或者 redis 宕机时导致大量请求直接访问数据库，导致数据库崩溃。
解决方案：数据库限流，集群部署使用主从 redis，当 redis 从机宕机时。立即可以有替代机器访问。

有没有接触过消息中间件

1. 有接触过 redis、kafka 和 mq，用来做异步队列。当用户发起请求之后，不要求实时得到最终结果的业务，都可以考虑使用异步队列，这样就能立马返回中间结果。
2. 你们系统有哪些地方用到了异步消息队列？
 - 2.1 秒杀活动有做，秒杀成功之后，不需要马上生成订单，只需要告诉他秒杀资格成功了。
 - 2.2 我们系统的下单过程也做异步队列，订单会先存储在消息队列中，然后系统再对消息队列的订单进行处理（修改用户积分，推送订单消息，修改实际库存，在数据库插入订单记录等等）。

消息队列（redis、Kafka、MQ）你需要关注哪些内容

生产者产生的数据，能准确的可靠的被消费者使用，而且不会被重复消费；
对于有顺序的消息还要关注消息的处理顺序。

web 应用中间件

nginx、tomcat、redis 这种处于用户和数据库之间的软件称为中间件。

*robotframework 自动化测试框架

RF 是 python 的一个可视化测试工具，由关键字驱动，可用于 ui 自动化和接口自动化测试。
使用起来很简单，对 python 语言基础要求极低，上手快。但是可扩展性低，没有手写 python 代码好。

python+locust 进行性能测试

python+locust 模块，可以进行接口的性能测试。（百度搜一下，学习成本和 python+requests 差不多，3 天学会）

*微服务架构

1.2018 年之后，前后端分离的项目越来越流行。前端页面从后端程序中脱离出来，前后端分开更明确。

2.java 后端开发工程师专注写接口程序。web 前端开发工程师专注写页面，且当后端接口没有开发出来时，前端系统可能会写 mock 接口（模拟接口）而不依赖于后端接口服务。

3.“前端页面”可以部署到 nginx 静态服务器，也可以部署到 node.js 服务端环境。

“后端微服务接口”可打成多个 war 包独立部署，一个 war 包就是一个微服务。微服务自带 web 服务器模块（自带 tomcat），是不需要放到 tomcat 的 webapps 目录下（如果要放进去，也是可以的）。

用户在前端页面上基于 http 协议直接发请求到服务器 java 接口（此时，页面发出的接口请求存在跨域问题）。

4.java 的微服务技术框架有 spring boot、spring cloud、dubbo(阿里巴巴贡献给 apache 组织的开源框架)等等。

微服务程序适合把一个大的单体程序分为好几个小型服务程序，小型服务程序通过接口来交互，适用大型项目。

5.spring cloud 基于 spring boot，spring boot 自带 tomcat 服务器。大概 2015-2018 开始发展流行。

PS：1、中间件 nginx 不仅可以作为代理服务器进行请求分发和负载均衡，nginx 还可作为页面的静态服务器。

2、nginx 还可以和 apache 服务器一样作为“静态 web 服务器”（存放 html 页面和图片）。

Tomcat 属于“动态 web 容器”（web 服务器）。

*Docker 和 k8s

Docker 是一个开源的应用容器（特殊的虚拟机），让开发者可以打包他们的应用以及依赖包到一个可移植的 docker 镜像中，然后发布到任何流行的 Linux 或 Windows 操作系统的机器上，也可以实现虚拟化。容器是完全使用沙箱机制。

k8s 全称 kubernetes，k8s 也是为容器服务而生的一个可移植容器的编排管理工具，越来越多的公司正在拥抱 k8s，并且当前 k8s 已经主导了云业务流程，推动了微服务架构等热门技术的普及和落地。

PS：在容器技术之前，开发用虚拟机比较多，比如 vmware，我们可以使用虚拟机在我们的操作系统中模拟出多台子电脑（Linux），子电脑之间是相互隔离的，但是虚拟机对于开发和运维人员而言，存在启动慢，占用空间大，不易迁移的缺点。

*分布式、集群、微服务

分布式：系统的一个模块作为一个子系统单独部署在不同服务器上，子系统之间通过接口交互。

集群：多台服务器部署了同一个系统。分布式和集群往往合在一起，叫分布式集群：集群的每个节点刚好是分布式系统。

微服务：系统的一个模块作为一个子系统部署在同一台服务器上，子系统之间通过接口交互。

微服务是特殊的分布式。

*os 模块

os 是 python 自带的操作系统模块 operation system，提供了创建文件夹方法、创建文件方法、删除文件方法、修改文件名方法、获取当前目录方法、跳转目录方法等。通过 os.system() 方法还能执行 cmd 或 shell(Linux)命令。

PS : sys 模块 : sys.argv 在命令行执行脚本时，可以从程序外部为其传参。

B 端、C 端产品

B 端产品 : business , 指面向企业的产品，比如企业微信、钉钉、OA 人事管理系统。

C 端产品 : customer , 指面向个人的产品，比如微信、抖音、小红书。

ERP、CRM 系统



*Saas 系统

Saas 系统是为企业提供的一种系统平台，企业可以在上面注册登录付费，就可以享受到对应的服务。

比如 ERP、CRM 等 Saas 系统。企业无需单独购买软硬件、建设机房、招聘 IT 人员，即可通过互联网使用 Saas 系统，为企业降低信息建设成本。

PS : lot : 物联网 (Internet of Things)

SCM 供应链系统 : supply chain management , 集原材料供应、生产、分销、零售与一体的流程管理系统。

MES 生产管理系统 : Manufacturing Execution System , 制造数据管理、计划排程管理、生产调度管理、库存管理、质量管理、采购管理、成本管理、生产过程控制。为制造企业打造一个扎实、可靠、全面、可行的制造协同管理平台。

*硬件测试会嘛

1. 硬件测试很简单的，自己平时生活也有接触过，喜欢观察，也很感兴趣。（相当于进厂，进去就能学会）
2. 软硬件结合的测试很有前途：自动售货机、送餐机器人、智能门禁、智能手表、安防监控、无人机、车载项目等。

PS : 万用表、示波器接触过吗？ : 这些在高中时候，都用过，做过实验的。万用表是测电压、电流、电阻的。

也可以回答：我主要做软硬件结合的软件测试，纯硬件测试（电路）有电子信息工程的人做。

示波器用来显示电信号的波形图，直流电显示为直线，交流电显示为正弦波形。

*软件测试和硬件测试的区别

1、测试目的不同

硬件测试的目的主要是保障硬件的可靠性，以及硬件和硬件的联接关系的正确性与准确性。

软件测试的目的主要是保证软件流程的正确性，以及正确的应用逻辑关系。

2、测试手段不同

硬件测试，主要针对硬件本身以及环境的测试，比如老化测试、寿命测试、故障率测试等。

软件测试，主要通过对软件的输入进行控制，从而达到不同的结果，通过输入输出的差异比较测试是否正确。

3、测试工具不同

硬件测试更多的是使用硬件进行，比如示波器、万用表等。

软件测试相对来说，用到的只是数据性的工具，或者软件。

4、测试结果的稳定性不同

硬件测试有可能在相同的条件下（如相同的温度），出现不同的测试结果

软件测试的输入相同的话，如果没有引入随机数据，则其输出是相同的。

PS：软件测试是站在用户使用角度来进行测试。硬件测试是站在电气特性角度来进行测试（比如仪表电压标准是 12v）。我们软件测试工程师做的软硬件结合的测试 或 车载测试，其实还是看成是软件测试。

*车载测试了解嘛

版本 1（推荐）：

车载测试主要有车机测试、仪表测试、adas 测试。在测试中比较常用的是 Canoe 工具，它可以创建 DBC 数据库、发送报文、收集 Log、编写 Capl 函数等。

车机测试：主要指中控屏（**安卓系统**）蓝牙、音乐、语音、导航、空调等 app 的测试。

仪表测试：主要有油量表、水温表、车速表、转速表、远近光灯以及相关的报警灯。

adas 测试：ADAS 驾驶辅助测试，因为它可以帮助人们，去降低一些事故发生的几率。例如 AEB 紧急制动的功能，当你使用它时，假如你车辆前方突然出现过往的行人，它就会帮你自动停止你的汽车。

PS：Capl 语言类似 C 语言，可用于 Canoe 工具的脚本编写。adas 驾驶辅助的升级版就是自动驾驶了。

目前 L2 级别的自动驾驶（有条件的自动驾驶）比较热门，L4 级别的全自动化驾驶不太靠谱，容易出现安全事故。

版本 2：我有朋友做过，很简单，他做的是汽车中控屏（车机）的一些多媒体和导航测试，也是安卓系统，测试 app 的一些功能和稳定性。这个和普通 app 测试类似。

PMP 项目管理认证

首先 PMP 是(Project Management Professional)我们常说的项目管理专业人士资格认证，PMP 认证是由美国 PMI 协会于上世纪发起的，是项目管理领域目前含金量最高的认证，比 MBA 更具价值。适用于项目经理能力提升，技术工作转型管理。而 CAPM(Certified Associate in Project Management)是项目管理专业助理师的英文缩写，是项目经理助理岗位。

附录

1. web、app、H5、小程序测试介绍

1. web 测试：

功能测试：刷新、回退、前进、翻页、页面加载、url

兼容测试：不同的浏览器内核进行测试（IE、chrome、Firefox）

性能测试：web 项目需监测响应时间、CPU、内存

2. H5 测试：

Web 要在电脑端测试，H5 可以在电脑端和手机浏览器测试或者 app 内部打卡测试

功能测试：刷新、回退、前进、翻页、页面加载、url、界面测试

性能测试：页面加载时间、弱网络环境和服务端的压测

兼容性测试：浏览器兼容、系统版本兼容、分辨率兼容

app 打卡的 h5 测试点：

（1）session 传输

（2）登陆

（3）url 信息

（4）页面适配

优势：

（1）跨平台，兼容性强；

（2）开发速度快，成本较低；

（3）迭代周期短；

（4）用户使用成本低即点即用；

（5）技术成本低。

劣势：

（1）运行速度慢，耗费网速，用户体验受限；

（2）调用移动底层硬件设备效果不是很好，无法本地保存用户数据；

用户黏度不高，关闭后用户可能已经忘记自己刚刚的操作；

（3）同其他语言编写的网页一样容易泄漏一些敏感数据。

3. app 测试:

界面测试: 分辨率, 屏幕尺寸, 兼容性

设备系统: Android/iOS 多少系统 (安卓还要看不同手机)

专项测试:

- (1) 干扰测试: 中断, 来电, 短信, 关机, 重启
- (2) 弱网测试: 2g、3g、4g, wifi 网络状态差以及丢包
- (3) 安装、更新、卸载

优势:

- (1) 较于其他两个技术类型 APP 可提供最佳的用户体验, 最优质的用户界面, 最华丽的交互;
 - (2) 每一种移动操作系统都需呀独立的开发项目, 针对不同平台提供不同体验;
- 相较于 H5 可节省带宽成本, 以独立的应用程序运行 (并不需要浏览器);
- (3) 能够更加便捷有效的利用移动硬件设备的底层功能, 可访问本地资源;
 - (4) 盈利模式明朗, 用户黏度高, 一旦拉新成功, 只要产品对用户有价值, 一般不会轻易弃用。

劣势:

- (1) 移植不同平台所耗资源较多, 成本高;
- (2) 迭代周期受限;
- (3) 用户使用成本较高, 必须啊手动下载安装原生 APP;

4. 小程序测试:

1. 小程序类型

程序分为三种版本类型: 开发版, 体验版, 正式版。开发版和体验版无需审核, 需要给微信号配置权限, 通过扫小程序的二维码才能访问。正式版需要通过微信审核流程, 也就是说, 在开发阶段, 产品还未成型开始, 无论你想怎么折腾, 微信都有办法知道。这可不像你在网上找了个框架或是工具, 在本地怎么玩都没人知道。微信小程序开发者工具使用之前就要扫码的。开发版和体验版的区别, 在于开发版小程序的二维码有效期比较短。项目中, 我们一般会准备三套环境。开发版访问测试环境, 体验版访问预发布环境, 正式版访问生产环境。

2. 真机上的调试模式

微信小程序在真机上给开发者提供了调试模式, 在真机上打开微信小程序之后, 可以看到右上角的三个点, 打开之后我们能看到有打开调试的字眼, 直接选择打开即可, 这里有一个重启小程序的过程, 再次扫一扫二维码进入就可以看到调试了。

3.小程序码的兼容性问题

目前小程序不支持直接分享朋友圈, 只能分享微信好友。所以很多小程序都采取了“曲线救国”的方式, 通过生成带有小程序码的图片, 用户可以退出小程序将图片发布到朋友圈。既然把小程序码作为图片的一部分, 就涉及到小程序码的位置, 尺寸, 还得不影响原有图片的美观, 生成的小程序码还需要是可识别的。这需要前端工程师费功夫做不同屏幕尺寸的适配。

基础库与客户端之间的关系。小程序的能力需要微信客户端来支撑, 每一个基础库都只能在对应的客户端版本上运行, 高版本的基础库无法兼容低版本的微信客户端。基础库更新时机。为了避免新版本的基础库给线上小程序带来未知的影响, 微信客户端都是携带 上一个稳定版的基础库发布的。在新版本客户端发布后, 再通过后台灰度新版本基础库, 灰度时长一般为 12 小时, 在灰度结束后, 用户设备上才会有新版本的基础库。因此在不同版本的微信上进行测试, 也是有必要的。

4.页面回退(页面路径最多只能十层)

在这个保留当前页面, 跳转到应用内的某个页面, 使用 `wx.navigateBack` 可以返回到原页面。在官方文档中, 有提示注意: 目前页面路径最多只能十层。如果开发在这里处理不当, 有可能出现超过十层回退, 则会导致白屏, 无法操作, 导致必须重新打开小程序才能正常工作。

优势:

- (1) 同 H5 相似, 依赖于微信, 即点即用, 用户使用成本低;
 - (2) 主要样式代码都封装在微信小程序里, 打开速度比 H5 快, 慢于原生;
- 可以调用更多的手机硬件功能例如, GPS, 录音等;
- (3) 开发, 维护成本较低。

劣势:

- (1) 由于受到微信限制, 微信小程序只有 2M 的大小, 无法开发大型小程序;
- (2) 小程序技术框架还不稳定, 开发方法常有更新, 微信小程序团队也在开发过程中, 导致短时间内经常要升级维护;
- (3) 不能跳转外链网址, 所以间接影响了小程序的开放性, 可能是想限制其他支付方式活功能接入;
- (4) 暂时不能直接分享到其他平台, 少了一个重要的推广方式;
- (5) 需要像 APP 一样审核上架, 这点比 H5 即做即发布要麻烦;
- (6) 小程序没有 push 功能无法推送消息, 并且没有独立的用户体系, 用完即走。

5. 小程序测试点:

1、权限测试

需要检查以下几种情况下微信用户访问的权限

1) 未授权微信登录小程序

未授权时, 一般使用一些业务功能的时候, 都会弹出提醒: 先授权再操作对应功能。或者在提交数据到后台的

时候，会提示补充相关身份信息才能提交成功

2) 已授权微信登录、访问小程序，意味着自己的微信账号可被小程序管理方所获取，自动以微信的身份行使业务操作权限，比如咨询、支付、数据查询等

3) 同一微信号在不同手机端登录授权查看数据权限

同一微信号在不同手机微信端授权登录同一小程序之后，所能查看的数据和操作的权限都应该是同步一致的

2、功能测试

1) 按功能模块测试

根据设计好的各个大类功能模块划分，然后再逐级细化，覆盖到每个功能尽可能全面的测试点

2) 按业务流程测试

小程序的业务

比如咨询、支付、播放、查询、下载。把各个功能点串联起来形成完整的业务流程来检查；同一个业务，可能有不同的路径来实现，每个路径都需要覆盖检查

3) 按数据流向测试

根据数据从某一端操作输入和输出流向，设计基于数据流的测试用例。

输出的数据也可能成为另外一端的输入，检查输入的数据是否按照代码逻辑执行正确的输出，是否数据发生异常（无法输入；有输入却无任何输出；输出不正确；多余的输出其他信息等）如管理后台修改了折扣，小程序提交订单时折扣是否对应变化

4) 同一功能不同的入口有效性的检查

小程序中在首页、列表页、详情页、其他的业务功能相关页面，都有可能存在同一个功能的入口。

5) 系统交互检查

一般而言，产生数据和功能交互变化的情况主要有这几个分类：

前台<-->前台、后台<-->后台、前台<-->后台。

3、兼容性测试

不同手机分辨率上的微信、安卓、ios 不同操作系统上的微信。

4、易用性测试

1) 导航

定位到页面某个模块所在位置

回到顶部 or 底部，导航条的收展，导航标签的文字是否容易理解

2) 功能入口

重要且常用业务的功能入口，是否在比较显眼的位置，业务操作过程是否便于大多数用户使用和查看

3) 上下层级进入返回

首页<-->列表页、列表页<-->详细页 、 首页<-->详细页。

不同层级之间的进入和返回实现是否有相应按键易操作

4) 字体、图片、动态交互效果

字体：标签、标题、内容、动态播放字体等

图片：轮播图、背景图、封面图、触屏产生的交互图

5) 测试功能时结合进入小程序的渠道（如进入小程序时有引导页，测试引导页是否正常显示时）

5.1) 通过搜索小程序进入

5.2) 通过分享小程序进入

5.3) 通过最近访问小程序进入

5.4) 通过微信公众号进入

5.5) 通过识别二维码进入